Smart Home Appliance Identification using Deep Learning

Arjun Ramphal[1], Darian Robert[1], Okuthe Paul Kogeda[1], Mluleki Bryan Sinqadu[1]

[1]School of Mathematics, Statistics and Computer Science, College of Agriculture, Engineering and Science, University of KwaZulu-Natal, Durban, KwaZulu-Natal, South Africa.

Contributing authors: 223005056@stu.ukzn.ac.za; 223118685@stu.ukzn.ac.za; KogedaO@ukzn.ac.za; SinqaduM@ukzn.ac.za;

Abstract

This study investigates whether known appliance classes can be identified from payload-independent engineered traffic features while retaining practical edge-inference cost. The public IoT Device Identification dataset was first audited because the publisher-supplied test file omitted one device class and exhibited substantial feature-distribution shift. The two source files were pooled, exact duplicates removed, and 1,877 unique observations repartitioned into 1,200 training, 301 validation and 376 held-out test cases. Training-only preprocessing retained 244 predictors. Five tuned neural architectures and a tuned Random Forest were evaluated under the same split. The Attention CNN–BiLSTM achieved **81.543 ± 1.227**% mean held-out accuracy across five seeds; the Feature-Attention MLP achieved **81.117 ± 0.921**%, and the tuned Random Forest achieved 83.511%. The validation-selected CNN–BiLSTM was the strongest selected neural model at 82.713% accuracy and 0.8158 macro-F1. Post-hoc analysis showed that `lights`, `socket` and `water_sensor` comprised 26.6% of held-out cases but 73.8% of the selected model's errors; collapsing them into one difficult-class group yielded 99.73% group-membership accuracy. Full-test Kernel and correlation-aware Partition SHAP showed strong feature-rank agreement (Spearman 0.8230; 60% top-10 overlap), with TTL, timing and packet-size/byte-volume signals recurring across both methods. Static-batch-one FP32 LiteRT profiling on x86_64 and ARM64 showed that the MLP and CNN had the lowest prepared-input latency, while the CNN–BiLSTM measured 3.848 ms and 3.979 ms, respectively. The results support closed-set behavioural identification from payload-independent traffic features, but show that class-level reliability, predictive quality and deployment cost must be considered jointly.

Keywords: Internet of Things, device identification, behavioural fingerprinting, CNN–BiLSTM, attention pooling, network-flow classification, edge inference

1 Introduction

1.1 Context

Consumer Internet of Things (IoT) devices have become common components of domestic networks. Smart cameras, monitors, sensors, sockets, thermostats, watches and other appliances communicate continuously with local gateways and cloud services. This creates a heterogeneous environment in which network administrators may not immediately know which class of device is responsible for a particular network flow. Reliable device identification can support network inventory, device-aware access control, segmentation, anomaly triage, troubleshooting and quality-of-service management. It has been demonstrated that statistical characteristics derived from network traffic can distinguish IoT devices without relying exclusively on application-payload content [1].

IP addresses can change through dynamic allocation, while MAC addresses may be spoofed, randomised or unavailable beyond the local network segment. Payload inspection also becomes less useful when application traffic is encrypted and can introduce privacy and implementation concerns.

Behavioural fingerprinting offers an alternative by characterising how a device communicates rather than examining the semantic content of its communications. Packet volumes, packet sizes, timing characteristics, protocol behaviour and other observable network statistics can collectively form a device-class fingerprint.

1.2 Problem Statement

Smart-home networks contain heterogeneous devices whose classes cannot always be established reliably from dynamic network addresses or encrypted application payloads. The technical problem is therefore to identify known IoT appliance classes accurately using only payload-independent engineered network traffic features.

This poses a multi-objective classification problem. The selected model must distinguish appliance classes whose network behaviours may overlap, while avoiding evaluation practices that produce misleading performance estimates. At the same time, additional architectural complexity must be justified against its latency, memory, storage and implementation costs in edge-oriented computing environments [2]. The central problem addressed by this study is consequently whether a deep-learning classification framework can provide an effective balance between predictive performance and practical inference cost, and how that balance compares with simpler neural and classical machine-learning alternatives.

1.3 Aim, Research Questions and Objectives

The aim is to design, implement, and evaluate an automated deep-learning classification framework capable of identifying smart-home appliances from payload-independent engineered network traffic features.

The main research question is:

How can a deep learning classification framework be designed, utilised and optimised to accurately identify and classify smart-home appliances using payload-independent engineered network traffic features?

Three sub-questions guide the study:

1. What payload-independent engineered network traffic features can support behavioural fingerprints for diverse IoT devices?
2. Which deep-learning architectures are most effective for identifying IoT devices from payload-independent engineered network traffic features, and how can an effective architecture be designed and implemented using robust data preprocessing and optimisation techniques?
3. How can the implemented classification framework be evaluated in terms of predictive performance and edge viability using controlled compute infrastructure?

The three corresponding objectives, aligned respectively with the sub-questions, are to:

1. Identify and characterise payload-independent engineered network traffic features that can support behavioural fingerprints while addressing dataset integrity and preprocessing leakage.
2. Comparatively evaluate relevant deep-learning architectures and design, implement and optimise an effective classifier using robust preprocessing and hyperparameter optimisation.
3. Evaluate the resulting framework using predictive metrics and controlled profiling of inference latency, memory utilisation and model footprint on x86_64 and ARM64 CPU infrastructure.

1.4 Scope and Contributions

This study investigates closed-set classification of ten smart-home appliance categories using payload-independent engineered network-traffic features. It identifies device classes rather than individual devices and neither inspects payloads nor detects unseen classes.

An Attention CNN–BiLSTM is compared with a Feature-Attention MLP, standalone CNN, LSTM and BiLSTM models, and a tuned Random Forest on a common held-out partition. SHAP provides post-hoc evidence about the selected CNN–BiLSTM's predictions. Deployment evaluation measures prepared-input neural inference on controlled x86_64 and ARM64 CPUs; packet capture, flow construction, feature extraction and policy enforcement are outside scope.

The contributions are:

1. auditing and correcting dataset-integrity problems affecting ten-class evaluation;
2. leakage-controlled comparison of classical, feed-forward, convolutional and recurrent models under a common partition;
3. repeated-run evaluation of neural-model sensitivity to training seed;
4. complementary SHAP analyses of attribution stability and cross-method agreement; and

5. validated LiteRT conversion and cross-platform measurement of neural-model latency, memory overhead and storage footprint.

The study examines predictive and deployment trade-offs in payload-independent appliance identification; it does not claim state-of-the-art performance across other datasets, generalisation to unknown devices or environments, or end-to-end gateway deployment.

Section 2 reviews related work and establishes the research gap. Section 3 describes the system design, implementation and evaluation procedure. Section 4 presents the predictive, interpretability and deployment results and limitations. Section 5 gives the conclusions and future work.

2 Literature Review and Related Work

2.1 Statistical Behavioural Fingerprinting

Research on IoT identification has progressively shifted from static identifiers toward traffic-derived fingerprints [3]. Sivanathan et al. classified devices in smart environments using network traffic characteristics, establishing that device behaviours can be distinguished from statistical properties of network communication without relying exclusively on application payload content [1]. Miettinen et al.'s IoT Sentinel demonstrated automated device-type identification for security enforcement and highlighted the operational value of identifying devices when they join a network [4]. Related approaches have further investigated automated identification using network-traffic characteristics and flow-level information, demonstrating the feasibility of passive device profiling [5, 6].

A broader survey by Safi et al. examined IoT profiling, fingerprinting, and identification techniques and identified traffic-related characteristics such as packet size, timing, protocol behaviour, and communication patterns as important sources of device fingerprints [3]. More recent survey work similarly categorises IoT identification approaches across the physical, network, and application layers and highlights continuing challenges involving heterogeneous devices, privacy, scalability, and generalisation across environments [7]. These findings motivate the use of passive, network-level behavioural characteristics in the present study.

2.2 Payload-Independent and Encrypted-Traffic Identification

Payload-independent identification uses observable packet and flow characteristics when application traffic is encrypted, or payload inspection is undesirable. Nakhla et al. investigated encrypted wireless traffic using engineered features and conventional classifiers, including Random Forest and k-nearest neighbours. Their commercial IoT Wi-Fi camera testbed demonstrated high identification accuracy using a relatively small feature set with modest computational requirements. However, firmware updates and changing device behaviour can alter network fingerprints, limiting their stability over time [8].

Zhang et al. proposed SecFinder, a smart-home identification system that extracts packet-size statistics from fixed-size traffic windows and applies multi-stage flow classification. The authors consider privacy and computational overhead and report improvements in identification accuracy and latency within their evaluation setting [9].

These studies establish the feasibility of payload-independent identification and motivate evaluating predictive performance alongside computational cost. The present study compares several neural architectures with a tuned Random Forest under a common evaluation protocol, using engineered traffic features and assessing neural-model footprint and inference latency.

2.3 Deep Learning Approaches

Deep learning has been applied to IoT identification using payload-derived and statistical representations. Kotak and Elovici processed network-communication representations with a convolutional neural network and reported high device-identification performance [10]. Bkheet et al. used recurrent deep learning with LSTM for smart-home appliance identification, reporting 83.25% test accuracy [11]. Wang et al.'s IoT-Portrait combines a Transformer with incremental learning and reports identification rates of up to 99% [12]. These findings provide architectural context, but differences in datasets, label spaces, representations and evaluation protocols prevent direct performance comparison with this study.

The architectures offer complementary operations. Convolution learns local feature interactions, while LSTM uses gated memory to mitigate vanishing-gradient limitations and selectively retain and update information over an ordered input [13]. Bidirectional recurrence combines forward and reverse processing [14], and attention learns to weight relevant intermediate representations.

The proposed Attention CNN–BiLSTM combines these operations to investigate their usefulness for engineered traffic features. However, the input follows a fixed predictor order rather than chronological packet observations. Recurrence therefore integrates information across the engineered-feature representation, and attention weights positions within the resulting representation. Any predictive improvement must be interpreted within this setting rather than as evidence of learned packet-level temporal dynamics.

2.4 Traditional Machine-Learning Baselines

Deep learning should not be assumed to outperform strong tabular machine-learning baselines. Tree-based ensemble methods can model nonlinear relationships and interactions among heterogeneous predictors without requiring the sequential assumptions imposed by recurrent architectures. This makes them important comparators for classification problems based on engineered network-traffic features.

Hussain et al. applied Random Forest and XGBoost-based methods to the same Kaggle IoT Device Identification dataset and reported 91% accuracy together with 93% precision, recall, and F1 in their experimental protocol [15]. This is an important comparator because it demonstrates that the dataset is highly amenable to tree-based decision boundaries. It also means that the value of a deep architecture must be established empirically rather than inferred from architectural sophistication.

Recent encrypted-traffic research provides further evidence that conventional machine-learning models can remain highly competitive. Nakhla et al. achieved strong results using engineered encrypted-traffic features and conventional classifiers rather than relying on deeper architectures [8]. This reinforces the importance of evaluating deep learning against an appropriately tuned classical baseline.

Accordingly, Random Forest is used as a tuned tabular baseline, a Feature-Attention MLP as a neural comparator, and standalone CNN, LSTM, and BiLSTM models as component baselines. This allows the hybrid architecture to be evaluated against both conventional machine learning and simpler neural alternatives rather than assuming that greater architectural complexity implies better performance.

2.5 Critical Comparison of Prior Approaches

The literature demonstrates several established approaches to IoT identification. Statistical and flow-based fingerprinting provide a passive means of distinguishing devices using observable communication behaviour, while deep-learning methods offer the possibility of learning more complex representations. Classical machine-learning approaches remain competitive when the input consists of engineered tabular traffic features. Recent studies have additionally examined encrypted traffic, privacy-preserving identification, computational efficiency, and changes in device behaviour caused by firmware or configuration changes [7–9].

Reported accuracy values should nevertheless be interpreted carefully. The reviewed studies differ substantially in datasets, numbers and types of devices, traffic-generation environments, feature representations, classification objectives, and evaluation protocols. A high accuracy reported for one dataset therefore does not establish equivalent performance for another. In particular, studies based on raw packet sequences, wireless-specific traces, active probing, or different device populations should not be treated as directly comparable benchmarks for the present ten-class engineered-feature task.

2.6 Research Gap

Traffic-based IoT identification using encrypted and payload-independent features is established, as is the feasibility of computationally efficient classification. The research addressed here is an integrated assessment of predictive reliability, explanation consistency and deployment cost on the selected engineered-feature benchmark. This assessment is necessary to determine whether additional architectural complexity provides sufficient predictive benefit to justify its computational requirements.

The study combines benchmark-integrity auditing and leakage-controlled reconstruction with comparison of tuned neural architectures and Random Forest on a common held-out partition. Training-only preprocessing preserves the evaluation boundary. Repeated Kernel SHAP estimates and correlation-aware Partition SHAP assess explanation stability and agreement for the selected CNN–BiLSTM, while FP32 LiteRT profiling quantifies latency, memory overhead and model footprint on x86_64 and ARM64 CPUs.

The contribution is a controlled comparative framework connecting dataset validity, predictive performance, interpretability and inference cost. Its findings support model selection within the investigated setting without claiming state-of-the-art classification accuracy.

3 Methodology and Experimental Procedure

3.1 System Architecture

Figure 1 summarises the workflow from dataset acquisition and leakage-controlled preprocessing to model development, evaluation and deployment profiling. The internal CNN–BiLSTM architecture is shown separately in Figure 3.

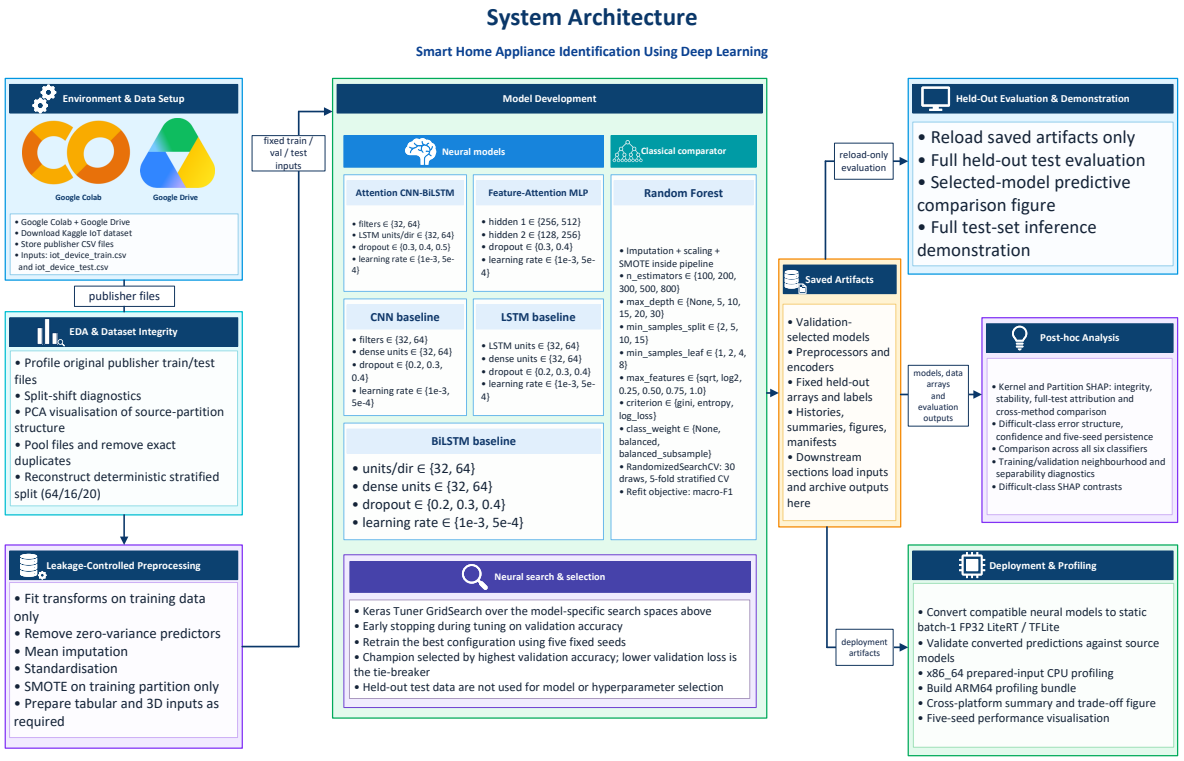


Fig. 1 Implemented architecture for data acquisition, integrity auditing, leakage-controlled preprocessing and model development using Google Colab and Google Drive. Saved models, preprocessing objects and fixed held-out arrays support reload-only evaluation, SHAP, validated FP32 LiteRT conversion and prepared-input CPU profiling on x86_64 and ARM64. Results and trade-off figures are archived.

The system consumes publisher CSV files containing engineered flow attributes. Selected models, fitted preprocessors, and fixed held-out arrays are saved so that evaluation, SHAP and conversion can be repeated independently of transient notebook state. Outputs proceed only after passing their stage-specific integrity, selection or equivalence checks.

3.2 Dataset Integrity and Partition Reconstruction

The public Kaggle "IoT Device Identification" dataset contains a 1,000-row training file and a 900-row test file, each with 297 predictors and the categorical target `device_category` [16]. Neither file contains missing values. Together they represent ten categories: TV, baby monitor, lights, motion sensor, security camera, smoke detector, socket, thermostat, watch and water sensor. A pre-modelling integrity audit found the publisher-supplied boundary unsuitable for closed-set ten-class evaluation.

Each class occurs 100 times in the training file, but `water_sensor` is absent from the test file. Removing 137 training-constant predictors left 160 active numeric predictors. Kolmogorov–Smirnov (KS), unequal-variance Welch and Mann–Whitney U tests compared their empirical distributions, means and rank distributions, respectively. Benjamini–Hochberg correction was applied separately to each family of 160 tests at $\alpha = 0.05$, leaving 111 KS, 108 Welch and 83 Mann–Whitney results significant.

Effect magnitude used the pooled scale

$$s_j^* = \sqrt{\frac{s_{\text{train},j}^2 + s_{\text{test},j}^2}{2}}, \qquad d_j = \frac{\bar{x}_{\text{train},j} - \bar{x}_{\text{test},j}}{s_j^*}, \qquad W_{1,j}^* = \frac{W_{1,j}}{s_j^*}.$$

Sixty-five predictors had $|d_j| \geq 0.2$, including 18 with $|d_j| \geq 0.5$ and nine with $|d_j| \geq 0.8$. The largest absolute effects were dominated by packet-size summaries, with `suffix_is_else` ranked first. The largest standardised Wasserstein distances were `suffix_is_else` (1.8436), `packet_size_A_max` (1.2009), `packet_size_A_stdev` (1.1334) and `packet_size_var` (1.0608). Agreement across distributional, mean, rank and magnitude diagnostics shows that the detected shift was not attributable only to statistically significant small effects.

Furthermore, 93 of the 137 training-constant predictors varied in the test file. A regularised logistic domain classifier tested whether all active predictors jointly revealed file membership [17]. Repeated 5-by-5 stratified cross-validation produced ROC-AUC 0.999962 ± 0.000055, and none of 200 random label permutations matched the observed five-fold score ($p = 0.004975$). Because ROC-AUC near 0.5 would indicate little separability, the near-perfect score demonstrates strongly encoded file membership. Together with the missing class and univariate results, this establishes substantial feature-distribution and class-support shift at the publisher boundary.

Five training rows and 18 test rows were duplicates. Pooling and exact-row deduplication removed these 23 observations, leaving 1,877 unique cases. Figure 2 summarises the largest KS differences and missing-class problem.

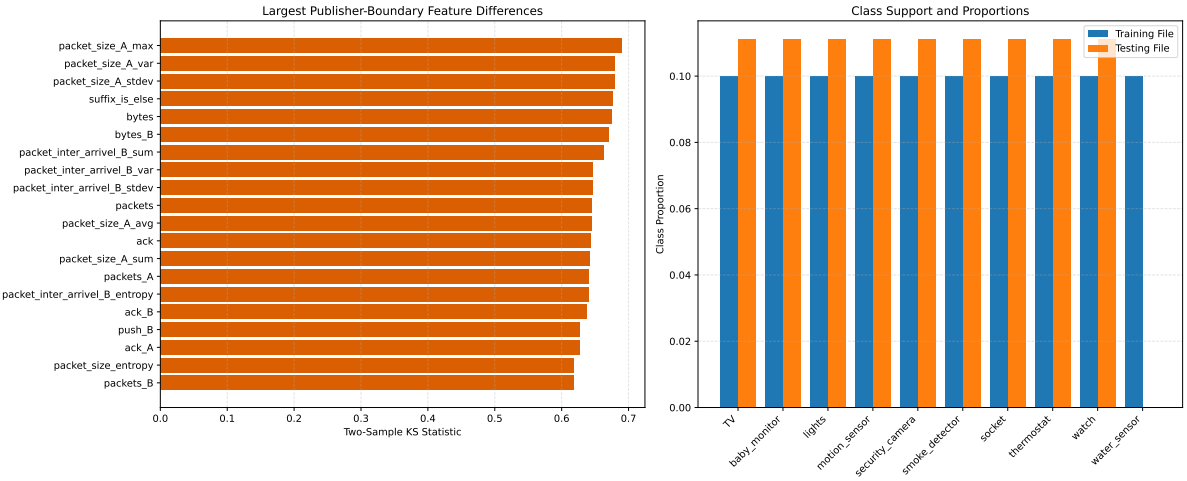


Fig. 2 The 20 largest KS statistics among the 160 active predictors and the missing `water_sensor` class in the supplied test file.

To retain all ten classes and reduce the anomalous publisher boundary's influence on architecture comparison, the unique pooled corpus was repartitioned. This avoids evaluating a ten-class model against a nine-class target space but sacrifices direct comparability with studies preserving the supplied split.

Using stratification and `random_state=42`, 20% of the 1,877 observations was held out for testing, and the remaining 80% was divided 80/20 into training and validation data. This produced 1,200 training, 301 validation and 376 test observations, with no exact feature vector shared across partitions. The partition was fixed before the post-split KS audit and was not regenerated or selected according to its outcome.

3.3 Leakage-Controlled Preprocessing

All learned preprocessing was fitted on training data only. Training variance removed 53 zero-variance predictors from all partitions, leaving 244 features. A mean imputer and `StandardScaler` were then fitted to the training partition and applied unchanged to validation and test data. The observed dataset contained no missing values, but retaining the imputer preserves a complete preprocessing pipeline. Labels were one-hot encoded.

SMOTE [18] with `random_state=42` was applied only after scaling and only to the training partition, increasing it to 1,280 observations. Validation and test cases remained real and untouched. All ten classes are represented in every reconstructed partition; the maximum class-proportion difference across partitions is 0.0026. A post-split audit found no train–validation or train–test predictor significant by KS testing after Benjamini–Hochberg false-discovery-rate correction [19] (0/244 in each comparison). This supports the absence of detected marginal imbalance without proving identical partition distributions.

3.4 Proposed Attention CNN–BiLSTM Model Architecture

The Attention CNN–BiLSTM tests whether local feature extraction, bidirectional integration and learned pooling improve identification. These operations follow the fixed engineered-feature order, not chronological packet time. Table 1 and Figure 3 present the selected forward path.

Keras Tuner GridSearch optimised validation accuracy over 24 combinations: filters and LSTM units per direction $\{32, 64\}$, dropout $\{0.3, 0.4, 0.5\}$ and Adam learning rate $\{10^{-3}, 5 \times 10^{-4}\}$. Trials used at most 50 epochs and validation-accuracy early stopping with patience 8 and restored weights. Selection yielded 64 filters, 64 units per direction, dropout 0.3 and learning rate 5×10^{-4}. The model contains 68,612 parameters: 68,228 trainable and 384 batch-normalisation moving statistics.

Table 1 Selected CNN–BiLSTM architecture; shapes omit the batch dimension.

Stage	Configuration	Output shape	Role
Input	244 imputed and standardised predictors; one channel	$(244, 1)$	Preserves predictor order and supplies the Conv1D channel dimension.
Conv1D	64 filters, kernel 3, ReLU, same padding	$(244, 64)$	Learns local nonlinear interactions without changing axis length.
Batch normalisation	64 channels	$(244, 64)$	Normalises activations using batch/moving statistics and learned scale/shift.
Max pooling	Pool size 2	$(122, 64)$	Retains local maxima and halves the feature-axis length.
Bidirectional LSTM	64 units per direction, `return_sequences=True`	$(122, 128)$	Concatenates forward and reverse states at each position.
Batch normalisation	128 channels	$(122, 128)$	Normalises recurrent states before dropout and attention.
Dropout	Rate 0.30	$(122, 128)$	Regularises training; inactive during inference.
Attention pooling	Trainable $W \in \mathbb{R}^{128\times1}$ and $b \in \mathbb{R}^{122\times1}$; softmax over 122 positions	(128)	Forms a learned weighted context vector.
Dense softmax	10 output units	(10)	Produces appliance-class probabilities.

For hidden vector $h_t \in \mathbb{R}^{128}$, shared trainable vector W and position-specific bias b_t, attention computes

$$e_t = \tanh(h_t^\top W + b_t), \qquad \alpha_t = \frac{\exp(e_t)}{\sum_{j=1}^{122} \exp(e_j)}, \qquad c = \sum_{t=1}^{122} \alpha_t h_t. \tag{1}$$

The weights sum to one, and the context $c \in \mathbb{R}^{128}$ feeds the ten-class softmax layer.

The selected configuration was trained with seeds 42, 7, 123, 2024 and 99 using categorical cross-entropy, Adam at 5×10^{-4}, batch size 32 and at most 100 epochs. Early stopping monitored validation loss with patience 15 and restored weights. `ReduceLROnPlateau` monitored validation accuracy with patience 5, factor 0.5 and minimum rate 10^{-5}. Highest restored validation accuracy, with lower validation loss as the tie-breaker, selected seed 42; held-out metrics did not influence selection. All five runs are reported individually and as mean ± sample SD, without assuming exact numerical repeatability across TensorFlow executions.

3.5 Comparative Models and Model Selection

The comparative models were chosen to determine whether the proposed hybrid provides useful predictive value relative to established tabular, feed-forward, convolutional and recurrent alternatives.

3.5.1 Tuned Random Forest Comparator

Random Forest [20] was the principal classical comparator because it models nonlinear tabular relationships without interpreting predictors sequentially, thereby testing whether neural architectures were necessary. The selected estimator averages leaf-probability vectors from 500 multiclass trees trained on bootstrap samples, with a random 50% of predictors considered at each split.

Mean imputation, zero-variance filtering, standardisation, SMOTE and the forest were enclosed in an `imblearn` pipeline, ensuring that every fitted operation was relearned within each cross-validation

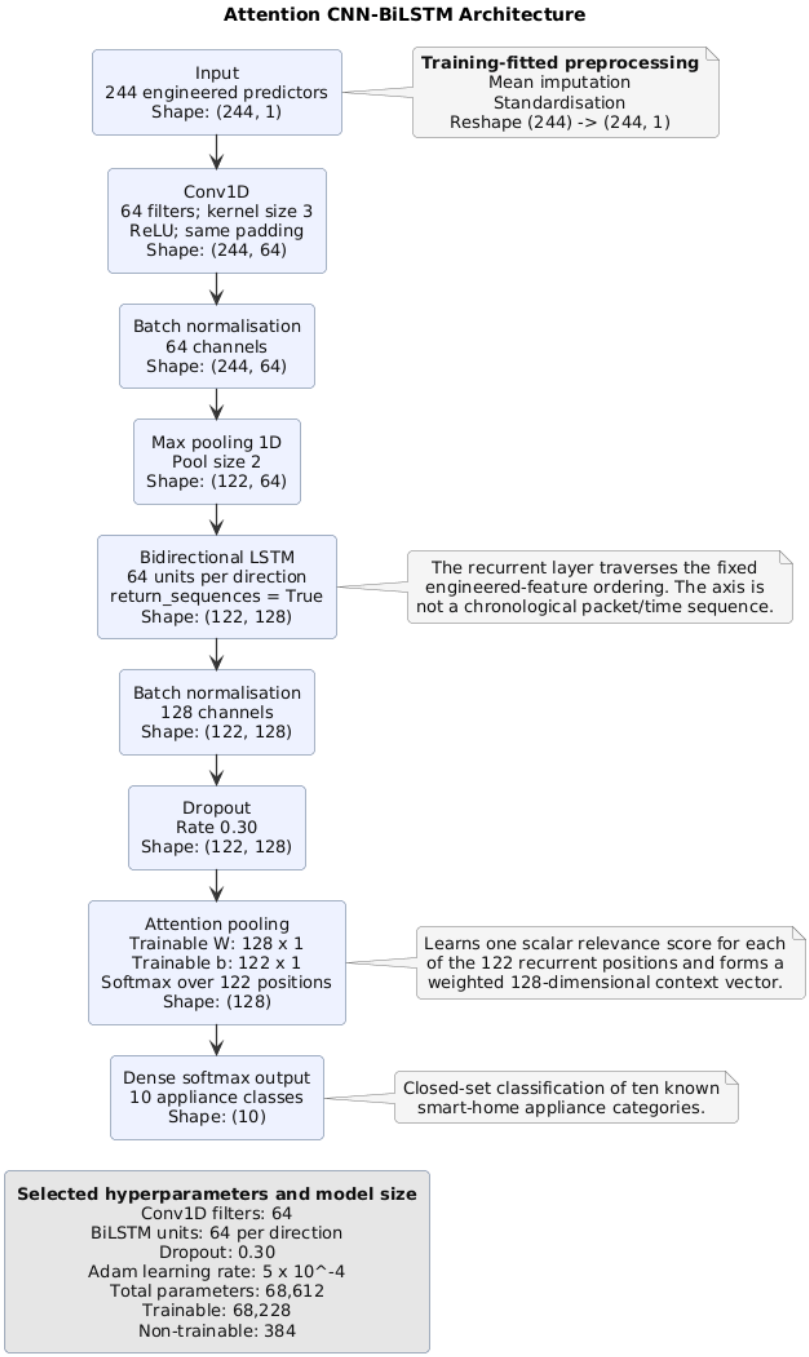


Fig. 3 Selected CNN–BiLSTM forward path. Convolution, recurrence and attention operate over engineered-feature order.

training fold. A 30-draw, five-fold stratified `RandomizedSearchCV` selected the pipeline by training-only macro-F1. The search varied the number of trees over $\{100, 200, 300, 500, 800\}$; maximum depth over $\{\texttt{None}, 5, 10, 15, 20, 30\}$; minimum split size over $\{2, 5, 10, 15\}$; minimum leaf size over $\{1, 2, 4, 8\}$; `max_features` over $\{\texttt{sqrt}, \texttt{log2}, 0.25, 0.50, 0.75, 1.0\}$; criterion over $\{\texttt{gini}, \texttt{entropy}, \texttt{log_loss}\}$; and class weighting over $\{\texttt{None}, \texttt{balanced}, \texttt{balanced_subsample}\}$. The selected `random_state=42` refit used 500 trees, unlimited depth, `min_samples_split`=10, `min_samples_leaf`=4, `max_features`=0.5, `log_loss` and `balanced_subsample`.

3.5.2 Feature-Attention MLP Comparator

The Feature-Attention MLP tested nonlinear tabular interactions without treating predictor order as temporal. It learns 244 softmax-normalised feature weights, applies them element-wise to the input and passes the result through ReLU layers of 256, 256 and 64 units. The first two layers use batch normalisation and dropout 0.4; the third uses batch normalisation, followed by a ten-class softmax

output. The selected model contains 207,694 parameters and tests whether direct feature weighting and dense transformations suffice without convolution or recurrence.

Keras Tuner GridSearch evaluated $2 \times 2 \times 2 \times 2 = 16$ configurations: first-layer width $\{256, 512\}$, second-layer width $\{128, 256\}$, shared dropout $\{0.3, 0.4\}$ and Adam learning rate $\{10^{-3}, 5 \times 10^{-4}\}$. Trials used at most 50 epochs, batch size 64 and validation-accuracy optimisation; early stopping had patience 8 and restored the best weights. The selected configuration used widths 256 and 256, dropout 0.4 and learning rate 10^{-3}.

This configuration was trained with seeds 42, 7, 123, 2024 and 99 for at most 100 epochs using batch size 64. Early stopping monitored validation loss with patience 15; `ReduceLROnPlateau` monitored validation accuracy with patience 5, factor 0.5 and minimum learning rate 10^{-5}. Seed 123 had the highest restored validation accuracy, with validation loss as the tie-breaker, and became the representative MLP.

3.5.3 CNN, LSTM and BiLSTM Component Baselines

These baselines isolate convolution, unidirectional recurrence and bidirectional recurrence from the proposed hybrid. All receive the 244 predictors reshaped as $(244, 1)$. The CNN models local interactions in the fixed feature order using 32 width-three valid-padding ReLU filters, width-two max pooling, flattening, a 64-unit ReLU layer, dropout 0.3 and ten-class softmax output; it has 248,650 parameters. The LSTM passes the final state of a 64-unit recurrent layer through a 64-unit ReLU layer, dropout 0.2 and softmax output; it has 21,706 parameters. The BiLSTM concatenates two 32-unit directional final states before the same 64-unit ReLU, dropout 0.2 and softmax stages; it has 13,514 parameters. The recurrent models traverse engineered predictor order, not packet-time sequences.

Each architecture used a 24-configuration validation-accuracy grid. The CNN crossed filters $\{32, 64\}$, dense width $\{32, 64\}$, dropout $\{0.2, 0.3, 0.4\}$ and Adam learning rate $\{10^{-3}, 5 \times 10^{-4}\}$; it selected 32 filters, 64 dense units, dropout 0.3 and learning rate 10^{-3}. The LSTM and BiLSTM replaced filters with recurrent units $\{32, 64\}$ while retaining the other choices. The LSTM selected 64 recurrent units, 64 dense units, dropout 0.2 and learning rate 5×10^{-4}; the BiLSTM selected 32 units per direction, 64 dense units, dropout 0.2 and learning rate 10^{-3}. Trials used at most 50 epochs, batch size 32 and validation-accuracy early stopping with patience 8 and restored weights.

Each selected configuration was trained with seeds 42, 7, 123, 2024 and 99 for at most 100 epochs using batch size 32, categorical cross-entropy and Adam at its selected learning rate. Early stopping monitored validation loss with patience 15 and restored the best weights; `ReduceLROnPlateau` monitored validation accuracy with patience 5, factor 0.5 and minimum learning rate 10^{-5}. Validation-only selection chose seed 42 for the CNN and LSTM and seed 123 for the BiLSTM.

3.6 Implementation Environment and Reproducibility

Tables 2 and 3 record the model-development hardware, software versions, implementation rationale and retained reproducibility artifacts. The deployment-profiling hosts and controlled profiling protocol are described separately in Section 3.10 because those configurations directly affect the reported latency and memory measurements.

Table 2 Recorded model-development environments.

Environment	Recorded configuration	Use
Google Colab–neural runs	Linux 6.6.122+ x86_64; Intel Xeon 2.00 GHz; one physical/two logical CPUs; approximately 12.67 GiB RAM; NVIDIA Tesla T4 with 15,360 MiB	Neural tuning and five-seed training, held-out evaluation, SHAP and conversion. The GPU accelerated training; recorded training time is not used as a cross-platform benchmark.
Google Colab–Random Forest	Linux 6.6.122+ x86_64; Intel Xeon 2.20 GHz; one physical/two logical CPUs; approximately 12.67 GiB RAM; no GPU available	Fold-isolated randomised search, final pipeline fitting and artifact generation on CPU.

Together, the retained artifacts allow held-out evaluation, model conversion, explanation post-processing and deployment profiling to be repeated from saved outputs without retraining the models in the same Colab session.

Table 3 Software components, selection rationale and retained reproducibility evidence. The complete package environment is recorded in run-specific requirements files.

Component	Implementation and rationale	Retained evidence/artifact
Execution and storage	Python/Colab for accessible managed computation; Google Drive for persistence across sessions	Run timestamp, terminal output, executed source, source hash, `runtime_environment.json`, `system_diagnostics.json`, `requirements.txt` and file inventory
Deep learning	TensorFlow 2.20.0, Keras 3.13.2 and Keras Tuner for required layers, custom attention, search and export	Source hash, search/training histories and `.keras` artifacts
Tabular pipeline	NumPy 2.1.3, pandas 2.2.3, scikit-learn 1.6.1 and imbalanced-learn 0.14.2 for explicit transformations, fold isolation and metrics	Fitted imputer/scaler, retained-feature lists, split tensors, labels and metric files
Classical model	scikit-learn Random Forest and Joblib 1.6.0 for a strong nonlinear tabular comparator and complete-pipeline serialisation	Fitted pipeline, search results, selected parameters and held-out probabilities
Explainability	SHAP 0.52.0 and SciPy 1.16.3 for model-agnostic and correlation-aware attribution checks	Raw attribution tensors, fidelity/stability summaries and feature/family tables
Deployment	Static-batch-one FP32 LiteRT for portable TensorFlow-model inference without a full training runtime	Validated `.tflite` files, conversion comparison and per-process profiling JSON/CSV
Archiving	Timestamped run directories and SHA-256 inventories	Metadata, source, logs, figures and tamper-evident file inventory

3.7 Predictive Evaluation Metrics

Accuracy and macro-F1 are the principal comparison metrics. Balanced accuracy, macro and weighted precision/recall/F1, log loss, macro one-vs-rest ROC-AUC, macro average precision, confusion matrices and per-class results additionally expose class imbalance, probability quality and error structure.

Let N denote held-out observations, $K = 10$ classes, y_i the true class, $\hat{y}_i$ the prediction and p_{ik} the probability assigned to class k. Confusion-matrix rows are true classes and columns are predictions. Accuracy is

$$\text{Accuracy} = \frac{1}{N}\sum_{i=1}^{N}\mathbf{1}(y_i = \hat{y}_i). \tag{2}$$

For confusion matrix C, let $TP_k = C_{kk}$, $FP_k = \sum_{a\neq k} C_{ak}$ and $FN_k = \sum_{b\neq k} C_{kb}$. Then

$$P_k = \frac{TP_k}{TP_k + FP_k}, \qquad R_k = \frac{TP_k}{TP_k + FN_k}, \qquad F1_k = \frac{2P_kR_k}{P_k + R_k}, \tag{3}$$

with zero used for an undefined ratio. For $Q_k \in \{P_k, R_k, F1_k\}$ and class support $n_k = \sum_b C_{kb}$,

$$Q_{\text{macro}} = \frac{1}{K}\sum_{k=1}^{K} Q_k, \qquad Q_{\text{weighted}} = \sum_{k=1}^{K}\frac{n_k}{N}Q_k, \qquad \text{Balanced Accuracy} = \frac{1}{K}\sum_{k=1}^{K} R_k. \tag{4}$$

Multiclass log loss measures true-class probability quality:

$$\text{LogLoss} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{K}\mathbf{1}(y_i = k)\log\big(\max(p_{ik}, \epsilon)\big), \tag{5}$$

where ϵ prevents $\log(0)$ and smaller values are better. Threshold-independent discrimination treats each class as positive against all others:

$$\text{ROC-AUC}_{\text{macro}} = \frac{1}{K}\sum_{k=1}^{K}\int_0^1 \text{TPR}_k(u)\,\mathrm{d}u, \tag{6}$$

where u is the false-positive rate. For successive recall–precision points $(R_{k,m}, P_{k,m})$,

$$\text{AP}_k = \sum_m (R_{k,m} - R_{k,m-1})P_{k,m}, \qquad \text{AP}_{\text{macro}} = \frac{1}{K}\sum_{k=1}^{K}\text{AP}_k. \tag{7}$$

All metrics for a selected model are derived from the same saved probability matrix.

3.8 Post-hoc Difficult-Class Diagnostic Analysis

After model and artifact selection, an explanatory post-hoc analysis examined the selected CNN–BiLSTM's weakest classes: `lights`, `socket` and `water_sensor`. It did not affect training, selection or reported held-out predictions.

All 376 decisions were reconstructed from the archived probabilities. The audit recorded error direction, winning probability, top-two probability margin, normalised predictive entropy and true-class rank. A reduced confusion matrix retained the three difficult labels and merged the remaining seven as *other*, distinguishing failure to recognise the difficult-class group from confusion within it. Saved probabilities from CNN–BiLSTM seeds 42, 7, 123, 2024 and 99 were used to recompute difficult-class precision, recall and F1 and count how many seeds misclassified each case. Corresponding metrics from the validation-selected MLP, CNN, LSTM and BiLSTM and the training-CV-selected Random Forest distinguished dataset-wide overlap from architecture-specific bias.

A training-and-validation-only data-space analysis recovered all 1,200 real and 80 synthetic observations from the shuffled post-SMOTE array through exact float32 multiset matching. Each synthetic observation's nearest real same-class and other-class Euclidean distances were compared. Validation observations were assessed using their nearest real training neighbour and the same-class proportion among their five nearest real neighbours. Logistic probes trained on real observations and evaluated on validation data measured pairwise linear separability for `lights`–`socket`, `lights`–`water_sensor` and `socket`–`water_sensor`; these were diagnostics, not additional comparative models. Training-only Mann–Whitney tests with Benjamini–Hochberg correction and rank-biserial effect sizes quantified univariate differences, while an exact-vector check identified retained feature vectors assigned conflicting labels.

3.9 Post-hoc SHAP Interpretability Analysis

SHAP was applied only after selection to interpret the chosen CNN–BiLSTM and did not affect training, tuning or held-out predictions [21]. It assigns feature contributions $\phi_{j,k}$ that approximately reconstruct class-k output from a background expectation:

$$f_k(x) \approx \mathbb{E}[f_k(X)] + \sum_{j=1}^{244}\phi_{j,k}(x). \tag{8}$$

Positive or negative values move the output above or below that expectation. These attributions explain the fitted model relative to its background and are not causal effects.

Kernel SHAP approximates Shapley values by evaluating masked feature coalitions against background observations and fitting a weighted local linear surrogate. It was selected as the primary method because it explains the complete custom model through its probability interface without architecture-specific gradient rules. Its limitations are computational cost and potentially implausible coalitions among correlated predictors. The same saved 244-feature preprocessing pipeline was used throughout. Its background comprised 20 real pre-SMOTE training observations nearest to $k = 20$ k-means centroids, providing representative regions without synthetic vectors.

The protocol had two stages. Stability was first assessed on a fixed class-balanced subset of 100 held-out cases, ten per class, using three independent 4,096-sample Kernel SHAP runs with seeds 1701, 2701 and 3701. Only this balanced audit was reweighted to the actual held-out class proportions. All-feature Spearman correlation, top-10/top-20 overlap and cosine similarity measured agreement. Final coverage then used one 4,096-sample run over all 376 held-out cases, requiring neither balancing nor reweighting.

Correlation-aware Partition SHAP provided a complementary sensitivity analysis. It follows a hierarchy of related predictors and assigns Owen values rather than using unrestricted coalitions. The hierarchy was derived with correlation distance from all 1,200 real pre-SMOTE training observations. Although this cannot guarantee physically realistic masked samples, it reduces arbitrary independent treatment of correlated engineered predictors. Partition SHAP used the same 376 cases, 20-observation background and 4,096-evaluation budget. For both full-test methods, global importance was the mean absolute predicted-class attribution across all cases. Local fidelity was checked by reconstructing predicted probabilities, and cross-method agreement by feature-rank Spearman correlation and top-10/top-20/top-50 overlap. Agreement strengthens method-independent conclusions; disagreement is retained as uncertainty.

Difficult-class follow-up reused the full-test tensors to explain specific confusion directions. For true class a, incorrect prediction b and feature j, the within-case contrast was $\Delta\phi_j = \phi_{j,b} - \phi_{j,a}$, with

$$D_j = \mathbb{E}[\Delta\phi_j \mid a \to b] - \mathbb{E}[\Delta\phi_j \mid a \to a]. \tag{9}$$

Positive values indicate features favouring the incorrect class more strongly in errors than in correct cases from the same true class. Kernel and Partition contrasts were calculated independently and compared through signed and absolute Spearman correlation and top-20 overlap. This associational analysis neither retrained the model nor interpreted contrasts as causal traffic mechanisms.

To limit over-interpretation of correlated predictors, all 244 features were also assigned through manually audited, mutually exclusive rules to packet-size/byte-volume, timing/inter-arrival, TTL, domain/suffix, SSL/TLS, TCP control/state, HTTP, port, directional-ratio, packet/transaction-count and IP-header families. Both total family share and mean attribution per member were considered because family sizes differ.

3.10 LiteRT Conversion and Controlled x86_64/ARM64 Profiling

The CNN–BiLSTM, Feature-Attention MLP, CNN, LSTM and BiLSTM were converted to static-batch-one FP32 LiteRT models. Where necessary, recurrent layers were rebuilt with CPU-compatible LSTMs, saved weights transferred, variables frozen and conversion restricted to built-in LiteRT operators. Acceptance required finite outputs, identical source/LiteRT predictions for all 376 held-out cases, unchanged accuracy and macro-F1, and probability differences within 10^{-5} absolute and relative tolerance.

The same protocol profiled all models on two CPU-only hosts. The x86_64 host was a Google Colab Linux 6.6.122+ worker exposing one physical/two logical Intel Xeon 2.00 GHz CPUs and approximately 12.67 GiB RAM, with Python 3.13.15 and LiteRT 2.2.0. The ARM64 host was an Oracle Cloud Ubuntu 24.04.5 KVM instance with four ARM Neoverse-N1 cores, approximately 23.4 GiB RAM, Python 3.12.3 and LiteRT 2.2.0. Workers were restricted to one logical processor (CPU 0 on ARM) and one LiteRT thread. Because the hosts differ in microarchitecture and virtualisation, results are platform-specific measurements rather than a controlled comparison of instruction-set architectures.

Each model was measured in three fresh processes. Every process performed 50 warm-ups before memory measurement and another 50 before timing. Memory was sampled every 2 ms across all 376 prepared inputs; latency used ten passes, producing 3,760 timed batch-one inferences. The timed boundary was prepared input → `set_tensor` → `invoke` → `get_tensor`, excluding packet capture, flow construction and feature extraction.

For latencies $\{\ell_1, \ldots, \ell_T\}$, median, P95 and P99 were the empirical 0.50, 0.95 and 0.99 quantiles. Peak memory overheads were

$$M_{\mathrm{USS}} = \max_t \mathrm{USS}(t) - \mathrm{USS}_{\mathrm{baseline}}, \qquad M_{\mathrm{RSS}} = \max_t \mathrm{RSS}(t) - \mathrm{RSS}_{\mathrm{baseline}}. \tag{10}$$

Generative AI Assistance

Generative AI assistance was used during the development of the research through OpenAI ChatGPT and Codex, primarily using GPT-5.6 Sol. Prompts supplied project requirements, relevant code excerpts, error traces, and draft report text. The tools assisted with literature-search suggestions, Python and Colab code revision, debugging and refactoring, figure code, and language editing. AI responses were treated as suggestions rather than empirical evidence. The authors selected the final methods and configurations, inspected and executed the code, and accepted numerical results only when reproduced from archived models, metrics, predictions, logs, environment records and figures. References were checked against official or publisher sources. No AI-generated data, fabricated measurements or unexecuted

results were reported, and the authors retain responsibility for the originality, correctness, interpretation and reproducibility of the work.

4 Results and Discussion

4.1 Five-Seed CNN–BiLSTM Performance

Table 4 reports the held-out results of the five authoritative CNN–BiLSTM training runs. Mean accuracy was $81.543 \pm 1.227\%$ and mean macro-F1 was 0.7928 ± 0.0157, where "±" denotes sample SD across the five observed runs. Accuracy ranged from 79.787–82.713%.

Table 4 Validation-selection and held-out performance across the five CNN–BiLSTM training seeds. Val. acc. denotes restored validation accuracy, bal. acc. denotes held-out balanced accuracy, ROC-AUC denotes macro one-vs-rest ROC-AUC, and AP denotes macro one-vs-rest average precision. Bold identifies seed 42, which was selected by the highest restored validation accuracy; held-out results were not used for selection.

Seed	Val. acc. (%)	Val. loss	Test acc. (%)	Bal. acc.	Macro-F1	ROC-AUC	Macro AP	Log loss
42	**81.728**	**0.4670**	**82.713**	**0.8172**	**0.8158**	**0.9823**	**0.8819**	**0.4076**
7	81.063	0.4856	82.713	0.8172	0.8002	0.9823	0.8850	0.4385
123	80.066	0.5039	79.787	0.7971	0.7764	0.9789	0.8608	0.4949
2024	80.731	0.4581	81.383	0.8247	0.7900	0.9800	0.8799	0.4357
99	78.405	0.4992	81.117	0.7970	0.7815	0.9796	0.8658	0.4397
Mean ± SD	80.399 ±1.265	0.4828 ±0.0199	81.543 ±1.227	0.8106 ±0.0128	0.7928 ±0.0157	0.9806 ±0.0016	0.8747 ±0.0107	0.4433 ±0.0318

Seed 42 was selected because it achieved the highest restored validation accuracy (81.728%) under the predefined validation rule. Its held-out accuracy was 82.713%, tied with seed 7 for the highest observed seed-level test accuracy. This agreement is descriptive only: the selection remained based exclusively on validation performance.

Figure 4 visualises the same five held-out seed-level accuracies and their descriptive mean ± sample SD. The tuned Random Forest is included only as a single-model reference line; it is not part of the CNN–BiLSTM seed distribution. The figure also highlights seed 42, which was selected by the predefined validation rule.

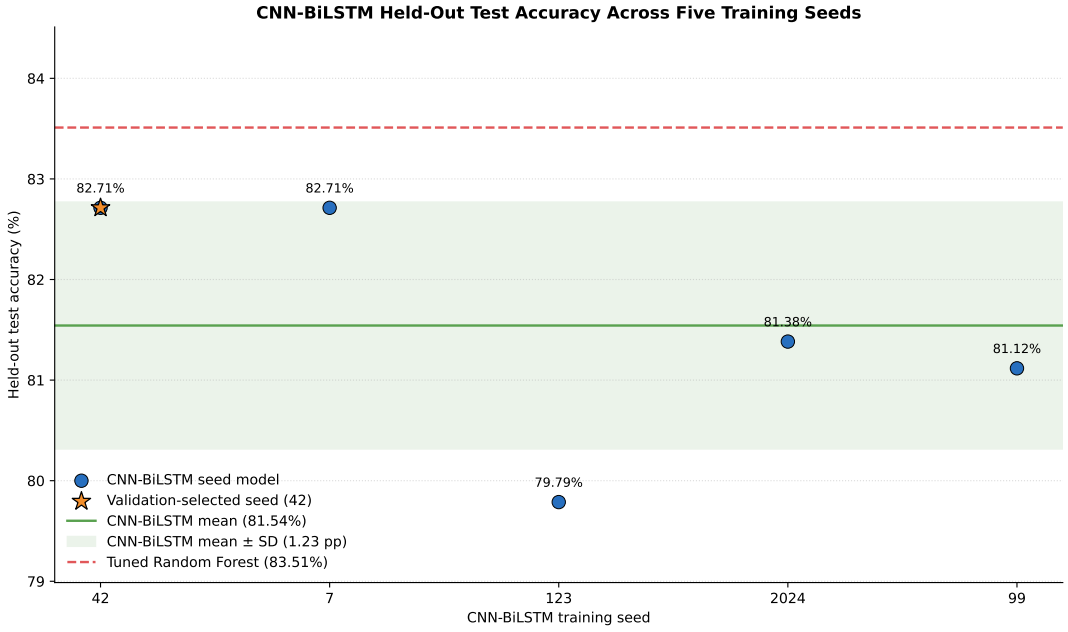


Fig. 4 Held-out test accuracy of the CNN–BiLSTM across seeds 42, 7, 123, 2024 and 99. The horizontal solid line shows the five-seed mean and the shaded band shows ±1 sample SD. The star identifies seed 42, which was selected using validation performance. The dashed tuned Random Forest line is a single-model reference rather than a like-for-like stochastic replication.

4.2 Comparison with Predictive Baselines

Table 5 reports validation selection and six held-out metrics for all five Feature-Attention MLP, CNN, LSTM and BiLSTM seeds. The Random Forest appears once because it is a single training-CV-selected refit rather than a replicated neural experiment. Its best training-only CV macro-F1 was 0.8532; after refitting, it achieved 85.05% validation accuracy and 0.8267 validation macro-F1.

Table 5 Validation-selection and held-out results for the neural baselines and tuned Random Forest. Neural validation entries report restored accuracy (%)/loss; bold identifies the validation-selected seed. Test metrics were not used for selection. FA-MLP denotes Feature-Attention MLP; bal. acc., ROC-AUC and AP denote balanced accuracy, macro one-vs-rest ROC-AUC and macro one-vs-rest average precision.

Model	Seed	Val. acc. / loss	Test acc. (%)	Bal. acc.	Macro-F1	ROC-AUC	Macro AP	Test log loss
FA-MLP	42	78.738 / 0.4211	80.585	0.8172	0.7782	0.9806	0.8750	0.4107
	7	82.060 / 0.3718	79.787	0.8095	0.7772	0.9806	0.8745	0.4987
	123	**82.724 / 0.3676**	81.915	0.8297	0.7953	0.9800	0.8741	0.3847
	2024	81.063 / 0.4205	81.383	0.8247	0.7884	0.9789	0.8749	0.4082
	99	82.724 / 0.3860	81.915	0.8146	0.7961	0.9807	0.8603	0.4030
	Mean		81.117	0.8191	0.7871	0.9802	0.8718	0.4211
	± SD	–	±0.921	±0.0081	±0.0090	±0.0008	±0.0064	±0.0446
CNN	**42**	**83.056 / 0.3956**	81.383	0.8247	0.7941	0.9759	0.8587	0.6143
	7	81.395 / 0.4822	78.457	0.7972	0.7648	0.9726	0.8491	0.6598
	123	80.731 / 0.4654	83.511	0.8197	0.8025	0.9824	0.8883	0.3919
	2024	82.060 / 0.4573	80.053	0.7872	0.7770	0.9746	0.8490	0.6340
	99	81.063 / 0.4333	80.585	0.8172	0.7784	0.9757	0.8674	0.6015
	Mean		80.798	0.8092	0.7834	0.9762	0.8625	0.5803
	± SD	–	±1.856	±0.0161	±0.0150	±0.0037	±0.0163	±0.1076
LSTM	**42**	**69.435 / 0.8919**	73.138	0.7242	0.7015	0.9464	0.7308	0.8979
	7	55.814 / 1.0431	56.915	0.5840	0.5383	0.9213	0.6211	1.0531
	123	64.120 / 0.9148	69.681	0.6808	0.6730	0.9521	0.7227	0.8797
	2024	64.784 / 0.9416	67.819	0.6668	0.6577	0.9478	0.7178	0.8605
	99	64.452 / 0.8957	65.426	0.6454	0.6377	0.9462	0.7240	0.8865
	Mean		66.596	0.6603	0.6416	0.9428	0.7033	0.9155
	± SD	–	±6.101	±0.0514	±0.0623	±0.0122	±0.0462	±0.0781
BiLSTM	42	70.100 / 0.8030	68.617	0.7027	0.6768	0.9623	0.7832	0.7419
	7	68.439 / 0.8479	68.883	0.6785	0.6687	0.9577	0.7544	0.7797
	123	**70.100 / 0.8027**	73.138	0.7189	0.7048	0.9656	0.8141	0.7213
	2024	60.465 / 0.9370	63.830	0.6165	0.6016	0.9478	0.7145	0.8548
	99	70.100 / 0.8540	75.266	0.7387	0.7284	0.9625	0.7677	0.6933
	Mean		69.947	0.6911	0.6761	0.9592	0.7668	0.7582
	± SD	–	±4.438	±0.0472	±0.0479	±0.0070	±0.0367	±0.0625
Tuned Random Forest	CV-selected	–	83.511	0.8347	0.8244	0.9835	0.8717	0.3922

Mean held-out accuracy was $81.117 \pm 0.921\%$ for the MLP, $80.798 \pm 1.856\%$ for the CNN, $66.596 \pm 6.101\%$ for the LSTM and $69.947 \pm 4.438\%$ for the BiLSTM. The CNN's advantage over the recurrent-only baselines supports convolutional processing of this engineered-feature representation, while their greater seed sensitivity reinforces that traversal of a fixed feature order is not chronological sequence modelling. Because capacities and search spaces differ, these results do not isolate any layer type as the sole cause.

Among the selected artifacts, the Random Forest achieved the highest accuracy (83.511%), macro-F1 (0.8244), balanced accuracy (0.8347) and macro ROC-AUC (0.9835). The CNN–BiLSTM was the strongest selected neural model by accuracy (82.713%) and macro-F1 (0.8158), and had the highest macro average precision (0.8819); the MLP had the lowest log loss (0.3847).

Figure 5 compares accuracy, balanced accuracy and macro-F1 for all five selected neural models and the Random Forest. Among neural models, the CNN–BiLSTM leads accuracy and macro-F1, the MLP leads balanced accuracy, and the CNN remains close to both. The lower LSTM and BiLSTM scores show that recurrence alone was less effective for the fixed feature ordering. The complete comparison therefore supports a multi-objective interpretation rather than automatic superiority of the proposed architecture.

4.3 Class-Level Error Structure

Table 6 shows that no classifier leads every class. The Random Forest leads `TV`, `socket`, `watch` and `water_sensor`; the CNN leads `baby_monitor` and `thermostat`; the MLP and CNN tie on

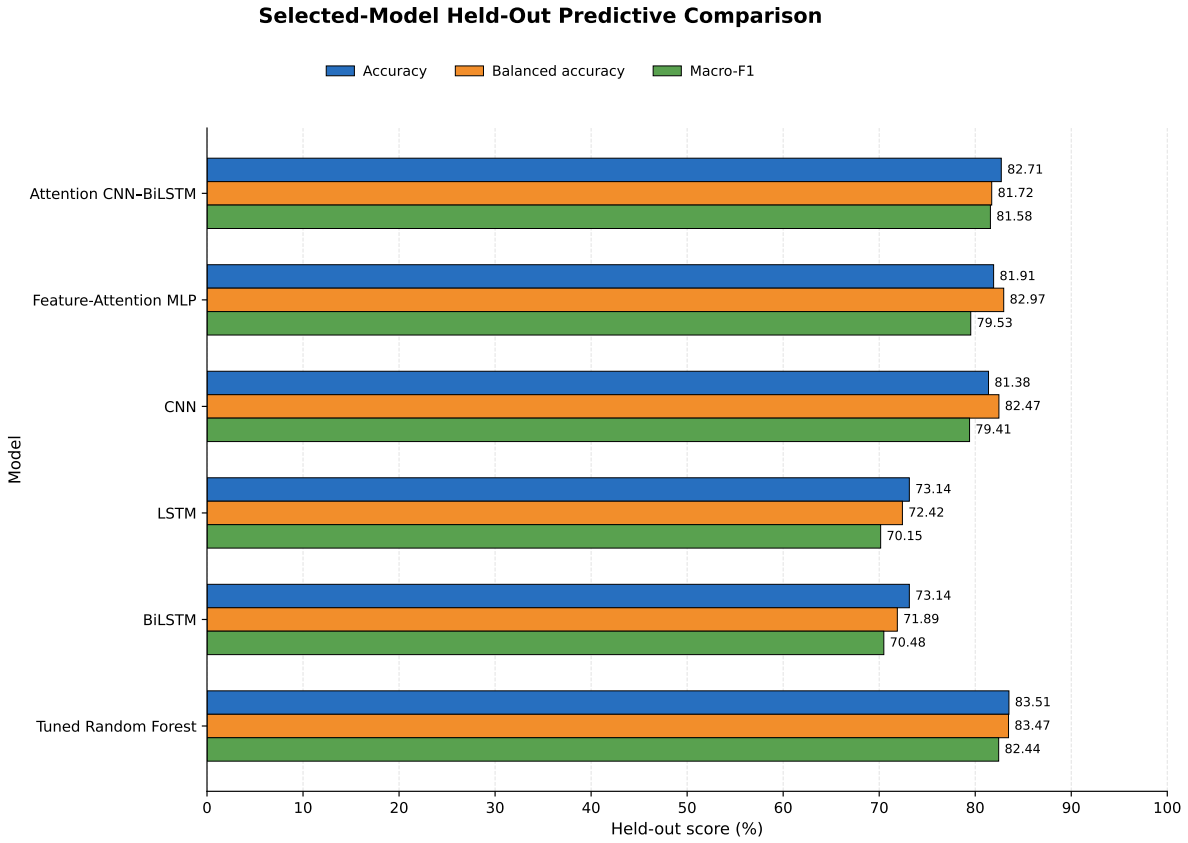


Fig. 5 Accuracy, balanced accuracy and macro-F1 on the common 376-case held-out partition. Neural artifacts were selected by validation performance and the Random Forest by training-only cross-validation.

`motion_sensor`; the CNN–BiLSTM leads `security_camera`; and the BiLSTM leads `lights`. All six models classify `smoke_detector` perfectly. Despite lower aggregate performance, the BiLSTM attains the highest light F1 (0.6446), while the LSTM and BiLSTM have particularly low socket F1 values of 0.2083 and 0.2128. The 20-case `water_sensor` support, compared with approximately 40 for most classes, makes its estimate less precise.

Table 6 Per-class F1 on the common held-out set. FA-MLP denotes Feature-Attention MLP; support is shared across models and bold marks row maxima, including ties.

Class	Support	CNN–BiLSTM	FA-MLP	CNN	LSTM	BiLSTM	RF
TV	40	0.9211	0.9250	0.8889	0.7229	0.4810	**0.9500**
Baby monitor	40	0.9630	0.9630	**0.9873**	0.8235	0.8533	0.9610
Lights	40	0.5556	0.5676	0.5634	0.6071	**0.6446**	0.4737
Motion sensor	39	0.9367	**0.9737**	**0.9737**	0.7879	0.8611	0.9315
Security camera	37	**0.9600**	0.9474	0.9231	0.8605	0.7907	0.9487
Smoke detector	40	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**
Socket	40	0.4651	0.2553	0.3265	0.2083	0.2128	**0.4865**
Thermostat	40	0.9268	0.9383	**0.9487**	0.8837	0.9250	0.9286
Watch	40	0.8718	0.8831	0.8354	0.5846	0.7342	**0.9367**
Water sensor	20	0.5581	0.5000	0.4938	0.5366	0.5455	**0.6275**

Figure 6 identifies the selected CNN–BiLSTM's principal confusion cluster. It correctly classifies 20/40 lights, misclassifying 17 as socket and three as water sensor; 20/40 sockets, misclassifying 12 as lights and eight as water sensor; and 12/20 water sensors, with the remaining eight labelled socket. The reciprocal errors concentrate on `lights`, `socket` and `water_sensor`.

4.4 Post-hoc Difficult-Class Error Analysis

The selected CNN–BiLSTM's errors are concentrated in `lights`, `socket` and `water_sensor`. These classes contain 100 of the 376 held-out observations (26.6%) but account for 48 of the model's 65 errors (73.8%). Their combined accuracy is 52.00%, compared with 93.84% for the other seven classes (Table 7).

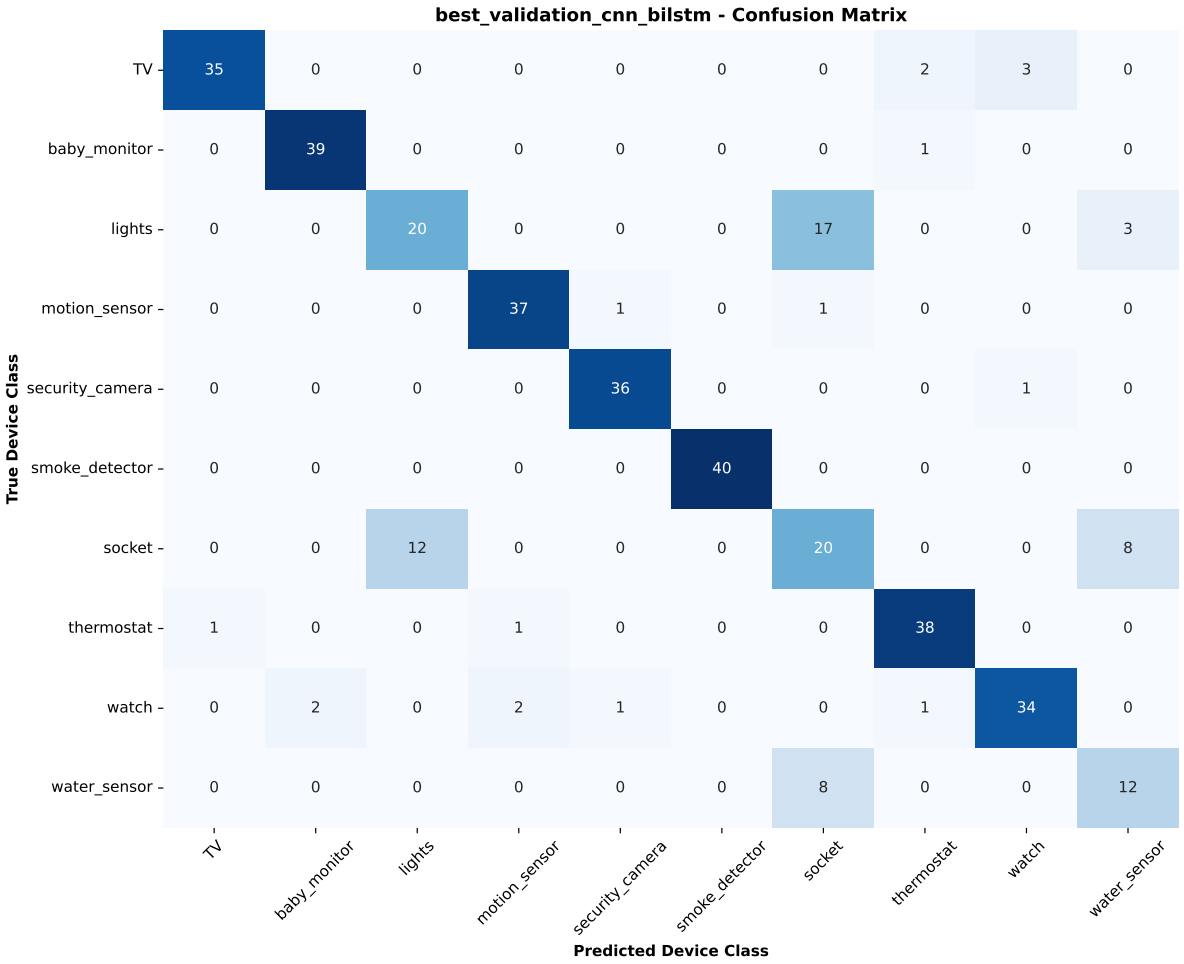


Fig. 6 Validation-selected seed-42 CNN–BiLSTM confusion matrix for the 376-case held-out set. Rows are true classes and columns are predictions.

Table 7 Error concentration for the validation-selected CNN–BiLSTM. The difficult group comprises `lights`, `socket` and `water_sensor`; error share is calculated over all 65 held-out errors.

Held-out group	Cases	Correct	Errors	Accuracy (%)	Error share (%)
Three difficult classes	100	52	48	52.00	73.85
Other seven classes	276	259	17	93.84	26.15
All classes	376	311	65	82.71	100.00

Every difficult-class observation was assigned to one of those three labels, while only one other observation, from `motion_sensor`, entered the group. Collapsing the three labels into one super-class therefore gives group-membership accuracy of $375/376 = 99.73\%$. The model distinguishes the group from the other devices but separates its members poorly.

Most within-group errors were uncertain: 31 of 48 had a winning-probability margin no greater than 0.05 and 43 no greater than 0.10. Only four had winning probability of at least 0.70 and two of at least 0.80. For the eight `water_sensor`→`socket` errors, mean winning probability was 0.3546 and mean margin 0.0105. Uncertainty-aware reporting could therefore mitigate many errors, although the few confident mistakes show that abstention would not resolve all of them.

4.4.1 Five-seed persistence and model-specific bias

The weakness persists across initialisations, although its direction varies. Across the five CNN–BiLSTM runs, `socket` had the lowest and most variable mean F1 (0.3112 ± 0.0952) and mean recall (0.2300). The validation-selected seed 42 achieved the highest observed socket F1, 0.4651, and therefore presents a more favourable result than the five-seed pattern. Twenty difficult-class observations were misclassified

by all five seeds – 19 sockets and one light – while 39 failed under at least four seeds and 48 under at least three.

Table 8 CNN–BiLSTM difficult-class performance across seeds 42, 7, 123, 2024 and 99. SD is the sample standard deviation over the five runs.

Class	Mean recall	Mean F1	F1 SD	F1 range
Lights	0.7500	0.6160	0.0600	0.5479–0.6786
Socket	0.2300	0.3112	0.0952	0.2174–0.4651
Water sensor	0.6900	0.5628	0.0406	0.4938–0.5882

Figure 7 treats seed identifiers as categorical observations rather than joining values with no ordinal relationship. It confirms persistent socket weakness and material variation in light and water-sensor performance.

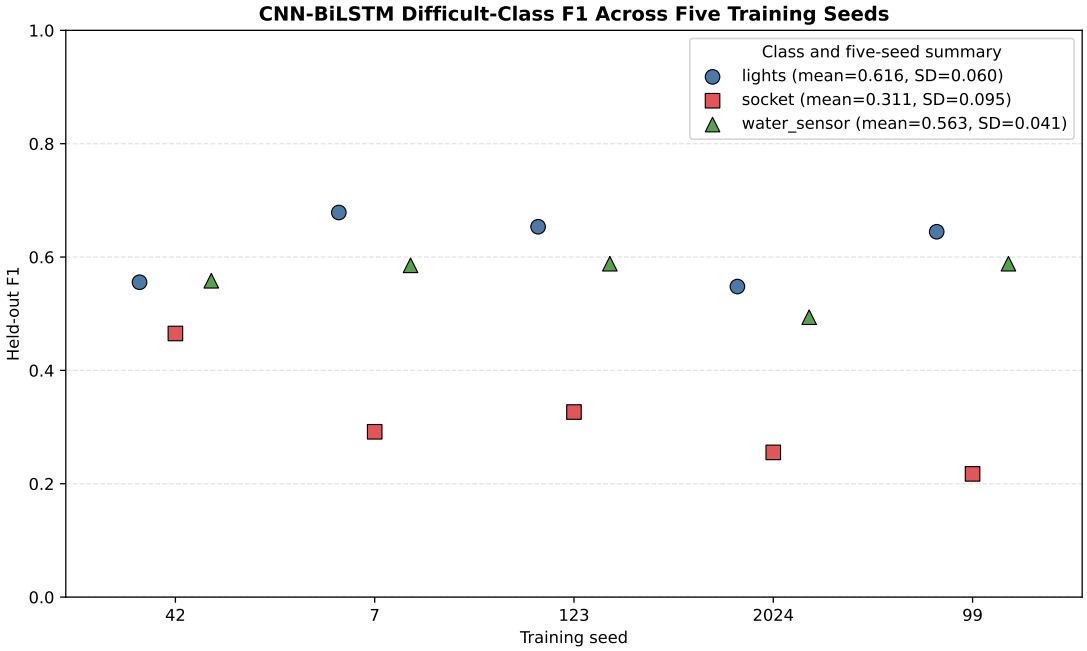


Fig. 7 Held-out F1 for the three difficult classes across five CNN–BiLSTM seeds. Seeds are categorical and are not joined; legend values report the mean and sample SD.

Table 6 shows that architectures express the overlap differently. The MLP and CNN recover every water-sensor case but often assign lights and sockets to that class, giving water-sensor precision of 0.3333 and 0.3279. The LSTM and BiLSTM favour `lights`: their light recall is 0.8500 and 0.9750, whereas socket recall is 0.1250 for both. The CNN–BiLSTM and Random Forest are more balanced. Their mean F1 across the three classes is similar – 0.5263 and 0.5292, respectively – but their class-specific differences indicate both shared feature overlap and architecture-specific decision bias.

4.4.2 Training/validation feature-space evidence

Before SMOTE, the training partition contains 127 real lights, 127 real sockets and 64 real water-sensor observations. Balancing each class to 128 creates one synthetic light, one synthetic socket and 64 synthetic water-sensor rows. Eighteen synthetic water-sensor rows (28.13%) are closer to a real observation from another class than to a real water-sensor observation. This does not establish that they are invalid, but places SMOTE within an overlapping local region.

Table 9 Training and validation diagnostics. The one-nearest-neighbour (1-NN) and five-nearest-neighbour (5-NN) calculations use real training observations as neighbours and untouched validation observations as queries.

Class	Real train	Synthetic	Validation 1-NN acc.	Mean same class among 5-NN
Lights	127	1	0.6875	0.5250
Socket	127	1	0.5938	0.5125
Water sensor	64	64	0.4375	0.3875

Water-sensor validation observations show the greatest local overlap, with the lowest 1-NN accuracy (0.4375) and mean same-class 5-NN fraction (0.3875). No retained feature vector carries conflicting labels, so the ambiguity is not caused by identical processed observations with different targets.

Real-training logistic probes achieved validation ROC-AUC of 0.7158 for lights versus socket, 0.8301 for lights versus water sensor and 0.6855 for socket versus water sensor. Training-only univariate tests found, respectively, 12, 98 and 82 of 244 predictors significant after Benjamini–Hochberg correction. Thus, useful but incomplete separation remains: lights versus socket has the weakest univariate evidence, while the modest lights–socket and socket–water-sensor probe results indicate that the representation does not provide clean linear boundaries.

4.4.3 Contrastive SHAP of observed error directions

Global SHAP agreement need not extend to small subsets representing specific errors. Table 10 compares Kernel and Partition estimates of D_j for the five observed within-group directions. Absolute-contrast rank correlation is moderate (0.3496–0.5624), with only 7–12 common predictors in each method's top 20. The `lights→water_sensor` result is particularly tentative because it contains only three errors.

Table 10 Agreement between Kernel and Partition SHAP error contrasts. Absolute Spearman compares the magnitudes of D_j across all 244 predictors.

Error direction	Cases	Signed Spearman	Absolute Spearman	Top-20 overlap
Lights → socket	17	0.2733	0.5624	7
Lights → water sensor	3	0.3226	0.3496	12
Socket → lights	12	0.2572	0.3727	8
Socket → water sensor	8	0.3547	0.4370	12
Water sensor → socket	8	0.4662	0.5150	8

Only recurring signals support robust interpretation. Both methods associate lights–socket errors mainly with packet-inter-arrival behaviour and `suffix_is_empty_char_value`. Water-sensor error directions repeatedly involve suffix/domain indicators, timing summaries, TTL statistics and `B_port_is_8080`. These associations suggest that errors occur when timing and observable service metadata resemble the competing class, but do not establish causality or transferability to other households or networks.

4.5 CNN–BiLSTM SHAP Stability and Full-Test Cross-Method Analysis

The fixed 100-case audit showed that Kernel SHAP was not dominated by Monte Carlo noise. Across three independent 4,096-sample runs, pairwise all-feature Spearman correlation was 0.9264–0.9375, top-10 overlap 90–100%, top-20 overlap 80–90% and cosine similarity 0.9958–0.9965 (Table 11). These results provide stability evidence; final global interpretation uses all 376 held-out cases.

Table 11 Pairwise stability of global predicted-class importance across three independent 4,096-sample Kernel SHAP estimates on the same 100-case subset.

Comparison	Spearman	Top-10 overlap	Top-20 overlap	Cosine
Rep. 1 vs 2	0.9375	90%	80%	0.9961
Rep. 1 vs 3	0.9361	90%	90%	0.9965
Rep. 2 vs 3	0.9264	100%	85%	0.9958

Kernel and correlation-aware Partition SHAP then explained all 376 held-out cases. Kernel SHAP had mean and maximum local reconstruction residuals of 1.98×10^{-8} and 8.94×10^{-7}; Partition SHAP achieved 4.07×10^{-9} and 6.91×10^{-7}. Their global rankings were strongly but not perfectly related: all-feature Spearman correlation was 0.8230, with top-10, top-20 and top-50 overlaps of 60%, 50% and 66%. Some redistribution is expected because Partition SHAP constrains correlated feature coalitions.

Six predictors appeared in both top 10 rankings (Table 12). Both methods ranked `suffix_is_else` first and `suffix_is_empty_char_value` fifth; three direction-A TTL summaries and `domain_is_else` completed the consensus set. Kernel SHAP additionally selected `ttl_B_entropy`, `reset`, `ssl_count_client_cipher_algs` and `B_port_is_else`, whereas Partition SHAP selected `reset_B`, `ttl_thirdQ`, `reset_A` and `ttl_A_min`. Recurring membership is therefore more reliable than the exact ordering of correlated predictors. Table 12 defines each consensus feature and its possible identification relevance.

Table 12 Predictors appearing in both full-test global top 10 rankings. Meanings describe the encoded predictors; identification relevance is interpretive rather than causal. K and P denote Kernel and Partition SHAP.

Predictor	Family	Statistical meaning	Possible identification relevance	K rank	P rank
`suffix_is_else`	Domain / suffix metadata	Indicates that the extracted suffix belongs to the residual `else` category.	May distinguish classes using domains or services whose suffixes are not retained as explicit categories.	1	1
`ttl_A_firstQ`	TTL statistics	First quartile of observed direction-A IP TTL values.	May reflect a repeatable combination of sender TTL defaults and traversed path length.	3	4
`suffix_is_empty_char_value`	Domain / suffix metadata	Indicates that the extracted suffix belongs to the dataset's encoded empty-character category.	May distinguish classes by whether usable suffix metadata is populated.	5	5
`domain_is_else`	Domain / suffix metadata	Indicates that the extracted domain belongs to the residual `else` category.	May capture class-specific use of domains not retained as explicit categories.	6	7
`ttl_A_median`	TTL statistics	Median of observed direction-A IP TTL values.	Summarises typical direction-A TTL and may reflect network-stack and path behaviour.	8	3
`ttl_A_thirdQ`	TTL statistics	Third quartile of observed direction-A IP TTL values.	Describes the upper TTL distribution and complements the first quartile and median.	10	10

Direction A remains a neutral label because the archived schema does not establish that it always means uplink, outbound or client-to-server traffic. TTL features contain values observed at the capture point rather than sender-initial TTL and can reflect both implementation defaults and route length. Domain and suffix indicators are one-hot metadata categories, not payload content, and may change with vendor cloud services, content-delivery infrastructure or DNS behaviour. These features therefore represent predictive associations rather than stable causal device signatures.

Figure 8 compares normalised attribution over the union of the leading features. Both methods emphasise suffix/domain indicators, TTL summaries, packet inter-arrival statistics and TCP state, while Partition SHAP concentrates more credit in correlated TTL and reset-related groups.

The same mutually exclusive signal-family mapping was applied to both attribution vectors. Family ordering was strongly concordant (Spearman 0.9364). Packet-size/byte-volume, timing/inter-arrival and TTL were the three leading families under both methods, and the same six families occupied the top six positions. Kernel SHAP assigned 28.63% of total absolute attribution to packet-size/byte-volume, 21.92% to timing/inter-arrival and 14.78% to TTL. Partition SHAP assigned 20.93%, 22.45% and 22.63%, respectively. Domain/suffix metadata contributed 6.59% under Kernel SHAP and 10.34% under Partition SHAP. After normalising for family size, TTL had the greatest mean attribution per predictor under both methods; SSL/TLS ranked second under Kernel SHAP and domain/suffix metadata second under Partition SHAP. Figure 9 presents the complete comparison.

Full-test analysis also improves class-conditional coverage: nine classes contribute 37–40 cases and `water_sensor` contributes 20. Interpretation nevertheless remains coupled to classifier performance. The selected CNN–BiLSTM correctly classified 20/40 socket, 20/40 light and 12/20 water-sensor cases, while six other classes had at least 34 correct predictions. Principal interpretability claims therefore rely on full-test global cross-method agreement rather than strong class-specific causal claims.

4.6 Cross-Platform LiteRT Profiling

All five neural models converted successfully, preserving 100% of held-out class predictions, accuracy and macro-F1. Table 13 reports all three fresh-process repetitions per model and platform. Both hosts used prepared-input batch-one inference, one LiteRT thread and processor affinity. After 50 excluded warm-ups, each process contributed 3,760 timed calls. Median, P95 and P99 latency and peak RSS/USS overhead are reported per process as x86_64/ARM64 pairs; file size appears once because the same FP32 LiteRT artifact was used in all six processes for each model.

Figure 10 combines selected-model held-out accuracy, deployment latency and file size. No model leads every dimension: the MLP has the lowest median latency, the CNN–BiLSTM the highest selected-neural accuracy, and the BiLSTM the smallest file but substantially weaker predictive performance. The

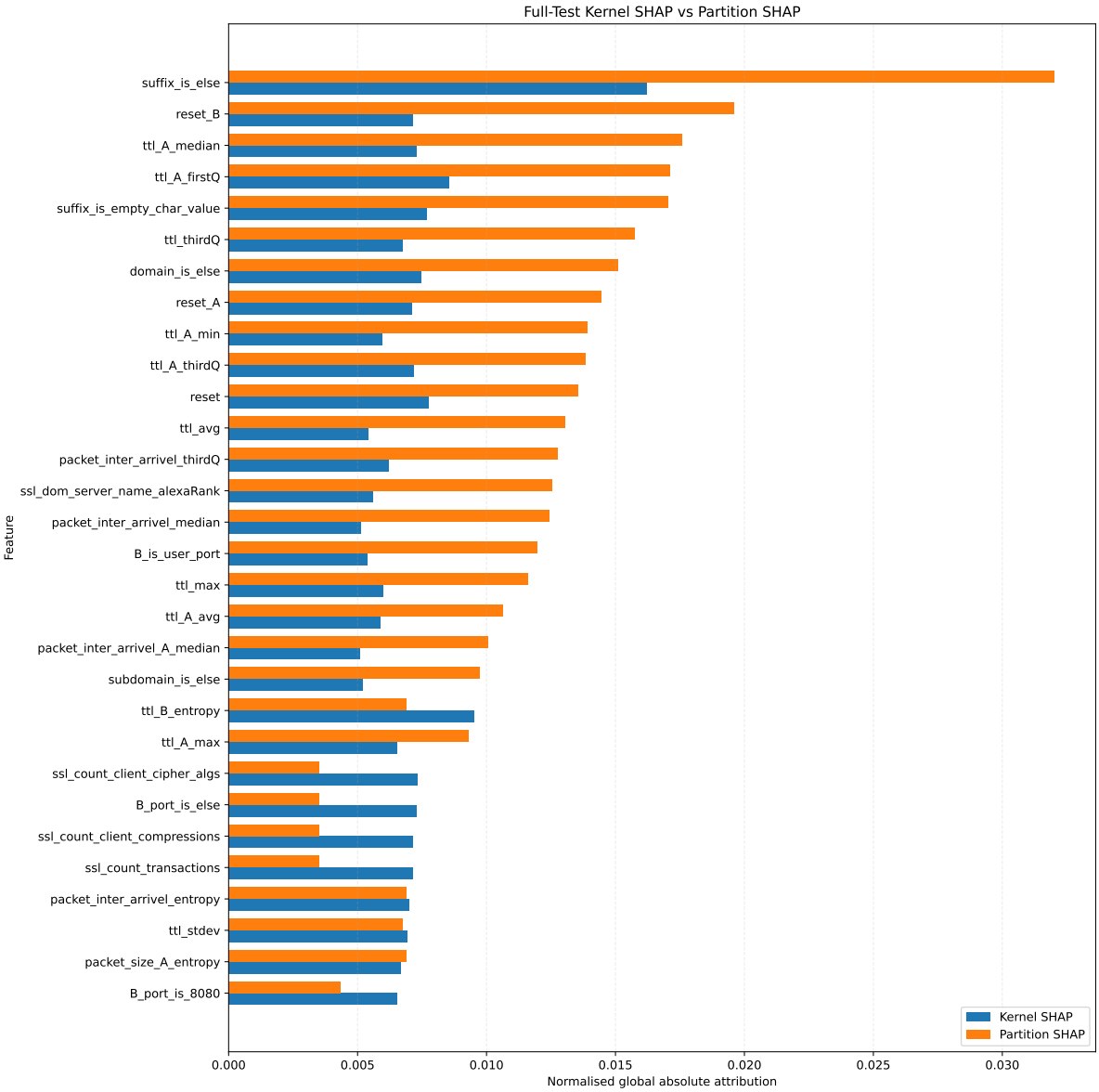


Fig. 8 Normalised global absolute predicted-class attribution over the union of both methods' top-20 features. Each method is normalised by its own total attribution; rank agreement uses the unnormalised importance vectors.

Table 13 Fresh-process FP32 LiteRT profiling on x86_64 and ARM64. Slash-separated values are x86_64/ARM64; latency is in milliseconds and RSS/USS are peak MiB above the fresh-worker baseline.

Model	Rep.	KiB	Median x86/ARM	P95 x86/ARM	P99 x86/ARM	Peak RSS x86/ARM	Peak USS x86/ARM
CNN	1	974.2	0.0421/0.0439	0.0680/0.0620	0.0868/0.0717	14.562/13.203	14.176/11.547
	2		0.0428/0.0436	0.0578/0.0628	0.0726/0.0763	14.562/13.199	14.176/11.543
	3		0.0390/0.0437	0.0658/0.0623	0.0861/0.0742	14.504/13.199	14.148/11.543
LSTM	1	95.7	0.8260/1.1158	1.0493/1.1457	1.4292/1.1579	12.809/11.539	12.352/9.879
	2		1.2286/1.1154	1.5567/1.1450	2.1540/1.1569	12.539/11.531	12.316/9.871
	3		0.8059/1.1141	1.3998/1.1432	1.5776/1.1571	12.738/11.539	12.391/9.879
BiLSTM	1	77.2	1.2525/1.2248	1.7020/1.2551	2.2363/1.2683	12.629/11.566	12.406/9.969
	2		1.2602/1.2193	1.7390/1.2495	2.2953/1.2630	12.617/11.566	12.383/9.969
	3		1.2682/1.2423	1.7100/1.2699	2.1970/1.2846	12.625/11.574	12.438/9.977
MLP	1	805.8	0.0244/0.0401	0.0384/0.0572	0.0628/0.0732	14.023/12.773	13.633/11.117
	2		0.0253/0.0396	0.0459/0.0558	0.0645/0.0665	13.965/12.773	13.613/11.117
	3		0.0244/0.0400	0.0369/0.0568	0.0460/0.0655	13.840/12.770	13.621/11.113
CNN–BiLSTM	1	333.3	3.8496/3.9083	6.5764/3.9557	7.7585/3.9851	14.152/12.887	13.770/11.230
	2		3.8080/4.0812	6.5149/4.1323	7.8210/4.1712	14.188/12.891	13.797/11.234
	3		3.8875/3.9470	6.3870/3.9981	7.4783/4.0264	14.145/12.887	13.758/11.230

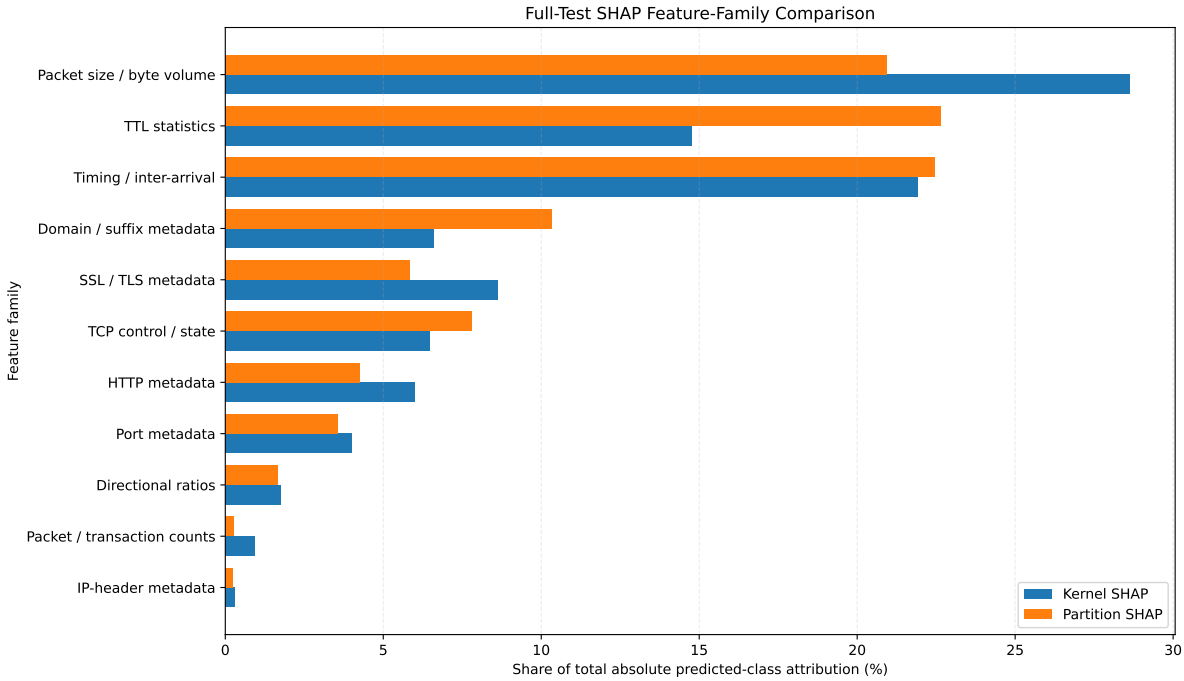


Fig. 9 Share of total absolute predicted-class attribution under the same mutually exclusive family mapping. Family totals depend on family size and are interpreted with per-feature importance.

CNN–BiLSTM is smaller than the CNN and MLP but has the greatest latency. These trade-offs are qualitatively consistent across both hosts.

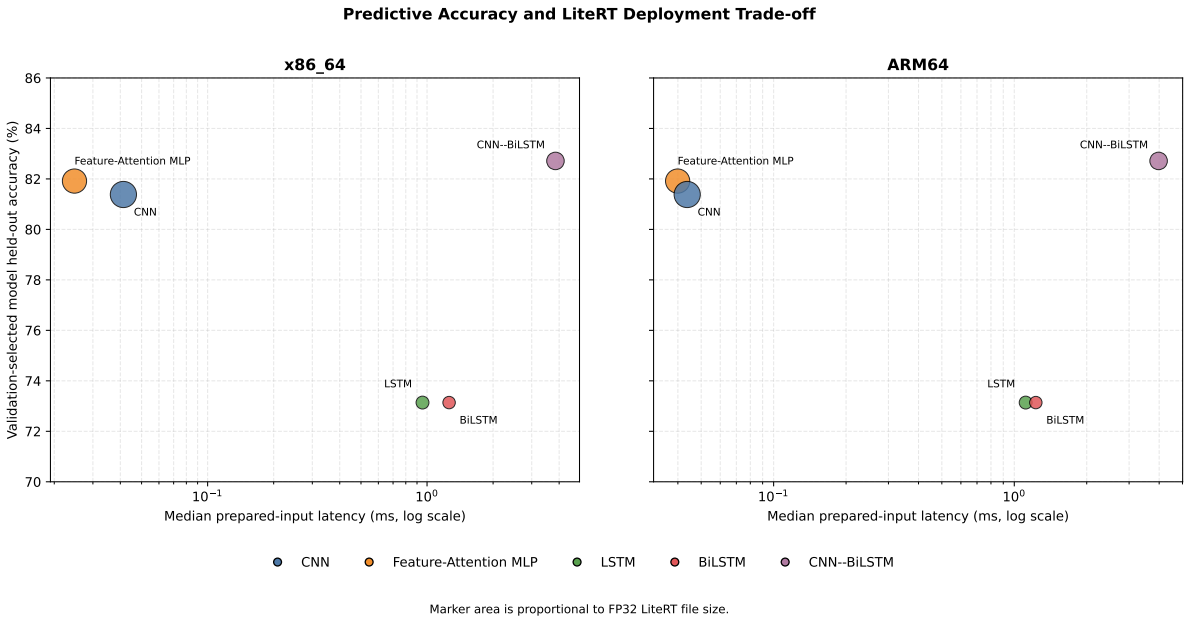


Fig. 10 Selected-model held-out accuracy versus median prepared-input batch-one LiteRT latency on x86_64 and ARM64. Marker area represents FP32 LiteRT file size, and latency is logarithmic. Only archived evaluation, profiling and model-size artifacts are used.

Average process-median latency produced the same order on both platforms: Feature-Attention MLP, CNN, LSTM, BiLSTM and CNN–BiLSTM. Individual peak USS overhead ranged from 12.316 to 14.176 MiB on x86_64 and 9.871 to 11.547 MiB on ARM64. Mean process-level peaks also gave the same memory order on both hosts: LSTM, BiLSTM, Feature-Attention MLP, CNN–BiLSTM and CNN. Because the x86_64 LSTM and BiLSTM intervals overlap slightly, this is a descriptive architecture-level ordering rather than strict process-level separation. The CNN–BiLSTM combined the highest selected-neural accuracy with a 333.3 KiB file, smaller than the MLP (805.8 KiB) and CNN (974.2 KiB), but was slowest at mean process-median latencies of 3.848 ms on x86_64 and 3.979 ms on ARM64.

Model choice is therefore objective-dependent. For predictive quality alone, the tuned Random Forest is preferred because it achieved the highest accuracy and macro-F1, although it was not profiled with LiteRT. For minimum neural latency, the MLP is preferred: it was fastest on both hosts and substantially more accurate than the recurrent-only baselines, at the cost of an 805.8 KiB file. The CNN is similarly fast but has the largest file and greater five-seed variability. The smaller LSTM and BiLSTM files do not offset their lower accuracy or provide minimum latency. The CNN–BiLSTM remains the principal neural model because it offers the strongest selected-neural performance, a moderate 333.3 KiB footprint and approximately 4 ms prepared-input inference, at the cost of the highest neural latency. It is therefore an accuracy–latency–storage compromise rather than an unconditional winner.

Consistent ordering across both environments suggests that the latency differences primarily reflect model structure rather than one benchmark run. Absolute x86_64 and ARM64 values are not intrinsic processor-architecture comparisons because the hosts differ in microarchitecture, virtualisation and configuration. The measurements also exclude packet capture and feature extraction, covering prepared-input LiteRT inference only.

4.7 Answering the Research Questions

RQ1 – What payload-independent engineered network traffic features can support behavioural fingerprints for diverse IoT devices? The answer is a distributed set of payload-independent signals rather than one defining variable. Kernel and correlation-aware Partition SHAP achieved 0.8230 all-feature rank correlation and shared six of their top-10 predictors, including the same rank-one suffix indicator and several TTL summaries. After predictors were grouped by signal type, both methods placed packet-size/byte-volume, timing/inter-arrival and TTL in the leading three families; domain/suffix, SSL/TLS and TCP-state information supplied additional evidence. The convergence of two attribution procedures and repeated Kernel estimates makes these feature-family conclusions more defensible than a single importance ranking. They remain statements about how the selected CNN–BiLSTM used the reconstructed dataset, not proof that the signals are causally necessary or guaranteed to remain stable in another household.

RQ2 – Which deep-learning architectures are most effective for identifying IoT devices from payload-independent engineered network traffic features, and how can an effective architecture be designed and implemented using robust data preprocessing and optimisation techniques?

Among the validation-selected neural models, the Attention CNN–BiLSTM achieved the highest held-out accuracy (82.713%) and macro-F1 (0.8158), followed by the Feature-Attention MLP and CNN; recurrent-only models were weaker. Its five-seed mean accuracy was $81.543 \pm 1.227\%$, indicating typical performance beyond the selected run. The tuned Random Forest remained the strongest overall predictive comparator.

The hybrid combined convolutional feature extraction, bidirectional integration over the fixed engineered-feature order, and attention pooling. Implementation used training-fitted variance filtering, imputation and standardisation, with SMOTE restricted to training data. Validation-based hyperparameter search, early stopping, learning-rate reduction and five seeded training runs supported optimisation. Representative models were selected using validation accuracy, with validation loss as the tie-breaker; held-out results did not influence tuning or selection.

RQ3 – How can the implemented classification framework be evaluated in terms of predictive performance and edge viability using controlled compute infrastructure? Predictive evaluation used one fixed 376-case held-out partition and complementary metrics for correct-class decisions, class balance, probability quality and class-level errors. Conversion testing then required all five neural LiteRT artefacts to preserve every predicted class, accuracy and macro-F1 within the declared probability tolerance. Controlled profiling used the same static-batch-one FP32 artefacts, fixed held-out tensors, one LiteRT thread, processor affinity and three fresh processes on both x86_64 and ARM64.

4.8 Limitations

Dataset and split. The deduplicated corpus contains 1,877 observations from ten known classes, so results may not generalise to other manufacturers, firmware, households, network conditions or unseen device types. Pooling the publisher files enabled balanced ten-class evaluation but limits direct comparison with studies retaining the supplied split.

Synthetic training data. Although restricted to training data, SMOTE may generate physically implausible high-dimensional traffic vectors. Development-only comparisons with random oversampling, class weighting, focal loss and no oversampling are therefore warranted.

Baseline optimisation. All neural models underwent validation-based hyperparameter searches, but their finite, architecture-specific search spaces were neither exhaustive nor compute-matched. The results therefore compare the selected configurations within those search spaces rather than the maximum performance achievable by each model family.

SHAP interpretation. SHAP explains associations within the fitted model rather than causal device behaviour, and attributions remain sensitive to background selection and correlated-feature handling. Conclusions are therefore limited to signals that remain consistent across Kernel and Partition SHAP.

Profiling scope. The x86_64 and ARM64 experiments used different virtualised hosts and measured only prepared-input LiteRT inference. Packet capture, flow construction, feature extraction, network I/O, sustained thermal behaviour and power draw were excluded, so the results do not represent end-to-end deployment.

5 Conclusions and Future Work

5.1 Conclusions

This study established that payload-independent engineered network traffic features can support closed-set identification of known smart-home appliance classes. An effective framework requires careful dataset auditing, training-only preprocessing and validation-governed model selection. The principal contribution is a reproducible evaluation that brings together predictive performance, explanation stability and deployment cost, providing evidence for assessing the practical value of different architectures.

The comparative findings show that architectural complexity must be justified empirically for this engineered-feature task. The Attention CNN–BiLSTM achieved the strongest performance among the validation-selected neural models, while the tuned Random Forest achieved the best held-out predictive results overall. The Feature-Attention MLP offered the lowest latency among the profiled neural models with competitive accuracy. These outcomes make model choice dependent on application requirements: the hybrid offers a compromise between neural predictive quality and model footprint, whereas the MLP is preferable when inference latency is the primary constraint.

The analysis also showed why aggregate accuracy alone is insufficient. Persistent confusion among lights, sockets and water sensors revealed strong recognition of the broader group but weaker discrimination between its members. Class-level reliability must therefore be considered before predictions are used to support device-aware network decisions. Agreement between repeated and complementary SHAP analyses strengthened the interpretation that packet-size, timing, TTL and observable metadata contributed to the selected model's decisions, while remaining model-specific associations rather than causal explanations.

Validated LiteRT conversion and profiling established portable prepared-input inference on the two evaluated CPU platforms. These measurements provide evidence of inference feasibility, while generalisation across households, recognition of unseen devices and complete capture-to-classification performance remain unestablished. The study therefore provides a controlled foundation for further development, with predictive quality, class reliability and computational cost jointly guiding model selection.

5.2 Future Work

Future work should evaluate a genuinely untouched household capture, open-set rejection of unknown devices, and temporal packet/flow windows that represent actual sequence structure rather than engineered feature order. The post-hoc evidence motivates a development-only comparison of SMOTE, random oversampling, class-weighted cross-entropy and class-balanced focal loss, with macro-F1 and difficult-class F1 selected before evaluation. A hierarchical neural design could first recognise the broader difficult-class group and then use a specialist head for `lights`, `socket` and `water_sensor`; this is motivated by 99.73% collapsed group-membership accuracy but remains untested. Because these hypotheses were generated from held-out errors, they should be assessed with out-of-fold training/validation predictions and repeated or nested cross-validation rather than selected on the existing test set. Further deployment work should measure the complete capture-to-classification path, sustained throughput, power and thermal behaviour on gateway-class hardware, and profile the Random Forest under a comparable runtime.

Academic supervision. This research was supervised by Prof. Okuthe Paul Kogeda (KogedaO@ukzn.ac.za) and Prof. Mluleki Bryan Sinqadu (SinqaduM@ukzn.ac.za), University of KwaZulu-Natal.

Author contributions. Arjun Ramphal: conceptualisation, methodology, software, validation, formal analysis, investigation, data curation, visualisation, writing–original draft, and writing–review and editing. Darian Robert: conceptualisation, methodology, software, validation, formal analysis, investigation, data curation, visualisation, writing–original draft, and writing–review and editing. Okuthe Paul Kogeda: supervision, methodology guidance, project administration, and writing–review and editing. Mluleki Bryan Sinqadu: supervision, methodology guidance, project administration, and writing–review and editing. Arjun Ramphal and Darian Robert contributed equally.

Funding. This research received no external funding.

Competing interests. The authors declare no competing interests.

Data and Code Availability. The source code, computational notebook, trained-model artefacts, aggregate experimental results, figure-generation sources, and reproducibility documentation are available from the public project repository at https://github.com/ArjunRamphal/smart-home-appliance-identification-using-deep-learning. The complete archived project structure and retained experimental artefacts are also available from the public Google Drive folder at https://drive.google.com/drive/folders/1_4Buza9xsp1shH3ZOvl8a-PHOVBsUwmt.

The dataset files originate from Chapter 5 of Packt Publishing's *Machine Learning for Cybersecurity Cookbook* repository: https://github.com/PacktPublishing/Machine-Learning-for-Cybersecurity-Cookbook. The repository is distributed under the MIT License. The Packt copyright and permission notice are retained with the redistributed dataset files. An equivalent copy is hosted on Kaggle as the *IOT Device Identification* dataset at https://www.kaggle.com/datasets/fanbyprinciple/iot-device-identification.

Appendix A: AI Declaration

Applicable to all student work

b) We made use of generative AI in this report/dissertation/thesis in the following areas: methodology implementation and review, data analysis, visualisation, literature-search suggestions, Python and Google Colab code revision, debugging and refactoring, figure code, and language editing. In all cases, the AI-generated content was fully interrogated for applicability, accuracy, and plagiarism. Citations were added to statements where applicable to credit authors.

c) Where generative AI was used in the methodology of this report/dissertation/thesis, this is indicated in the relevant methodology section, with full details enclosed.

Signed: Arjun Ramphal (223005056); Darian Robert (223118685)

References

[1] Sivanathan, A., Gharakheili, H.H., Loi, F., Radford, A., Wijenayake, C., Vishwanath, A., Sivaraman, V.: Classifying IoT devices in smart environments using network traffic characteristics. IEEE Transactions on Mobile Computing **18**(8), 1745–1759 (2019) https://doi.org/10.1109/TMC.2018.2866249

[2] Zhou, Z., Chen, X., Li, E., Zeng, L., Luo, K., Zhang, J.: Edge intelligence: Paving the last mile of artificial intelligence with edge computing. Proceedings of the IEEE **107**(8), 1738–1762 (2019) https://doi.org/10.1109/JPROC.2019.2918951

[3] Safi, M., Dadkhah, S., Shoeleh, F., Mahdikhani, H., Molyneaux, H., Ghorbani, A.A.: A survey on IoT profiling, fingerprinting, and identification. ACM Transactions on Internet of Things **3**(4), 26 (2022) https://doi.org/10.1145/3539736

[4] Miettinen, M., Marchal, S., Hafeez, I., Asokan, N., Sadeghi, A.-R., Tarkoma, S.: IoT SENTINEL: Automated device-type identification for security enforcement in IoT. In: 2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS), pp. 2177–2184. IEEE, Piscataway, NJ, USA (2017). https://doi.org/10.1109/ICDCS.2017.283

[5] Aksoy, A., Gunes, M.H.: Automated IoT device identification using network traffic. In: 2019 IEEE International Conference on Communications (ICC), pp. 1–7. IEEE, Piscataway, NJ, USA (2019). https://doi.org/10.1109/ICC.2019.8761559

[6] Chowdhury, R.R., Aneja, S., Aneja, N., Abas, P.E.: Network traffic analysis based IoT device identification. In: Proceedings of the 4th International Conference on Big Data and Internet of Things, pp. 79–89. Association for Computing Machinery, New York, NY, USA (2020). https://doi.org/10.1145/3421537.3421545

[7] Saadouni, C., El Jaouhari, S., Tamani, N., Ziti, S., Mroueh, L., El Bouchti, A.: Identification techniques in the internet of things: Survey, taxonomy and research frontier. IEEE Communications Surveys & Tutorials **28**(1), 593–632 (2026) https://doi.org/10.1109/COMST.2025.3541165

[8] Nakhla, N., Song, R., Vandenberghe, G., Islam, N., Watson, S., Bozorgi, M., Li, M.: Analysis of encrypted wireless traffic for identification of IoT devices. In: 2025 IEEE World AI IoT Congress (AIIoT), pp. 1107–1113. IEEE, Piscataway, NJ, USA (2025). https://doi.org/10.1109/AIIoT65859.2025.11105346

[9] Zhang, N., Chen, S., Liu, S., Deng, X., He, Y., Xiang, W., Li, M.: SecFinder: An IoT device identification system based on flow-level traffic in smart home. In: 2025 IEEE 31st International Conference on Parallel and Distributed Systems (ICPADS), pp. 1–8. IEEE, Piscataway, NJ, USA (2025). https://doi.org/10.1109/ICPADS67057.2025.11323115

[10] Kotak, J., Elovici, Y.: IoT device identification based on network communication analysis using deep learning. Journal of Ambient Intelligence and Humanized Computing **14**(7), 9113–9129 (2023) https://doi.org/10.1007/s12652-022-04415-6

[11] Bkheet, S.A., Agbinya, J.I., Khamis, G.S.M.: Advanced deep learning approach for smart home appliance identification using recurrent neural networks with LSTM. IoT **5**(4), 835–851 (2024) https://doi.org/10.3390/iot5040038

[12] Wang, J., Zhong, J., Li, J.: IoT-Portrait: Automatically identifying IoT devices via transformer with incremental learning. Future Internet **15**(3), 102 (2023) https://doi.org/10.3390/fi15030102

[13] Hochreiter, S., Schmidhuber, J.: Long short-term memory. Neural Computation **9**(8), 1735–1780 (1997) https://doi.org/10.1162/neco.1997.9.8.1735

[14] Schuster, M., Paliwal, K.K.: Bidirectional recurrent neural networks. IEEE Transactions on Signal Processing **45**(11), 2673–2681 (1997) https://doi.org/10.1109/78.650093

[15] Hussain, S., Aslam, W., Mehmood, A., Choi, G.S., Ashraf, I.: A machine learning based framework for IoT devices identification using web traffic. PeerJ Computer Science **10**, 1834 (2024). Article e1834. Available at https://peerj.com/articles/cs-1834/

[16] fanbyprinciple: IoT Device Identification. Kaggle dataset. Accessed 25 March 2026 (2021). https://www.kaggle.com/datasets/fanbyprinciple/iot-device-identification

[17] Rabanser, S., Günnemann, S., Lipton, Z.C.: Failing loudly: An empirical study of methods for detecting dataset shift. In: Wallach, H., Larochelle, H., Beygelzimer, A., d'Alché-Buc, F., Fox, E., Garnett, R. (eds.) Advances in Neural Information Processing Systems 32, pp. 1394–1406. Curran Associates, Inc., Red Hook, NY, USA (2019). https://proceedings.neurips.cc/paper/2019/hash/846c260d715e5b854ffad5f70a516c88-Abstract.html

[18] Chawla, N.V., Bowyer, K.W., Hall, L.O., Kegelmeyer, W.P.: SMOTE: Synthetic minority oversampling technique. Journal of Artificial Intelligence Research **16**, 321–357 (2002) https://doi.org/10.1613/jair.953

[19] Benjamini, Y., Hochberg, Y.: Controlling the false discovery rate: A practical and powerful approach to multiple testing. Journal of the Royal Statistical Society: Series B (Methodological) **57**(1), 289–300 (1995) https://doi.org/10.1111/j.2517-6161.1995.tb02031.x

[20] Breiman, L.: Random forests. Machine Learning **45**(1), 5–32 (2001) https://doi.org/10.1023/A:1010933404324

[21] Lundberg, S.M., Lee, S.-I.: A unified approach to interpreting model predictions. In: Guyon, I., Luxburg, U., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S.V.N., Garnett, R.

(eds.) Advances in Neural Information Processing Systems 30, pp. 4765–4774. Curran Associates, Inc., Red Hook, NY, USA (2017). https://proceedings.neurips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html